



دانشگاه دامغان

دانشکده فنی و مهندسی

پایان نامه دوره کارشناسی مهندسی کامپیوتر

سایت اشتراک گذاری API با پایتون

سجاد رضاقلی زاده

استاد راهنما:

دکتر زهره کریمی

تیر ۱۴۰۵

بسمه تعالی

تشر و قدردانی

از استاد راهنما و همه کسانی که در مسیر انجام این پروژه با راهنمایی، پیگیری و دلگرمی کنار من بودند، صمیمانه سپاسگزارم.

چکیده

این گزارش درباره یک پلتفرم فارسی برای اشتراک گذاری، جستجو و مصرف API است؛ سامانه‌ای که تلاش می‌کند تجربه کشف سرویس، خواندن مستندات، مدیریت حساب و انتشار API را در یک مسیر پیوسته کنار هم قرار دهد. در این پروژه، بک‌اند با Django REST Framework و رابط کاربری با React ساخته شده و فضای کلی آن به API Marketplace نزدیک است. کاربر می‌تواند کاتالوگ را ببیند، مستندات را بررسی کند، وارد حساب خود شود، مصرف و اشتراک را دنبال کند، API منتشر کند و از ابزارهایی مانند Caller و Studio کمک بگیرد.

گزارش از دو زاویه به پروژه نگاه می‌کند: تجربه کاربر و ساختار فنی. ابتدا روشن می‌شود کاربر با سامانه چه کاری انجام می‌دهد و سپس توضیح داده می‌شود این رفتارها چگونه میان رابط کاربری، بک‌اند و لایه داده تقسیم شده‌اند. منابع فنی از داکيومنت‌های رسمی مانند Django، Python، Docker، Node.js، React، Django REST Framework، OpenAPI و RapidAPI انتخاب شده‌اند. برای ارزیابی بک‌اند نیز دستور `python manage.py test api` اجرا شد و نتیجه آن در فصل آزمون آمده است.

کلیدواژه‌ها: پلتفرم اشتراک گذاری API، Django REST Framework، React، احراز هویت، OpenAPI

فهرست مطالب

د	فهرست علائم و نشانه‌ها
ه	فهرست جدول‌ها
و	فهرست نمودارها، عکس‌ها و نقشه‌ها
۱	۱ مقدمه
۱	۱-۱ زمینه پروژه
۱	۱-۲ بیان مسئله
۲	۱-۳ اهداف
۲	۱-۴ دامنه و محدودیت‌ها
۲	۱-۵ ذی‌نفعان
۳	۱-۶ سازمان‌دهی گزارش
۳	۱-۷ جمع‌بندی فصل
۴	۲ فناوری‌ها و پیشینه
۴	۲-۱ Django, Python و Django REST Framework
۴	۲-۲ MongoDB و مدل داده
۵	۲-۳ React, TypeScript و Vite
۵	۲-۴ Node.js و ابزارهای فرانت‌اند
۵	۲-۵ Docker Compose و اجرای محلی
۵	۲-۶ OpenAPI و RapidAPI
۶	۲-۷ جمع‌بندی فصل
۷	۳ تحلیل نیازمندی‌ها
۷	۳-۱ نقش‌های کاربری
۷	۳-۲ نیازمندی‌های عملکردی و غیرفعال
۹	۳-۳ موارد کاربرد سامانه
۱۱	۳-۳-۱ نمودارهای موارد کاربرد

۱۲	مورد کاربرد UC-01: کشف و جستجوی API	۳-۳-۲
۱۳	مورد کاربرد UC-02: بررسی جزئیات و مستندات	۳-۳-۳
۱۴	مورد کاربرد UC-03: ثبت نام، ورود و خروج	۳-۳-۴
۱۵	مورد کاربرد UC-04: مدیریت حساب، کلید و سازمان	۳-۳-۵
۱۶	مورد کاربرد UC-05: انتخاب و فعال سازی اشتراک	۳-۳-۶
۱۷	مورد کاربرد UC-06: انتشار API	۳-۳-۷
۱۸	مورد کاربرد UC-07: ثبت یا اصلاح امتیاز	۳-۳-۸
۱۹	مورد کاربرد UC-08: اجرای درخواست در Caller	۳-۳-۹
۲۰	مورد کاربرد UC-09: تعریف و ذخیره جریان در Studio	۳-۳-۱۰
۲۱	مورد کاربرد UC-10: ایجاد پروژه آغازین	۳-۳-۱۱
۲۲	مورد کاربرد UC-11: مشاهده داشبورد، دسترسی و مصرف	۳-۳-۱۲
۲۳	مورد کاربرد UC-12: مدیریت داده های زنده سامانه	۳-۳-۱۳
۲۴	مورد کاربرد UC-13: تحلیل آرشیو پروژه	۳-۳-۱۴
۲۵	مورد کاربرد UC-14: استقرار پروژه فشرده	۳-۳-۱۵
۲۶	مورد کاربرد UC-15: کار با کاتالوگ و پروژه از خط فرمان	۳-۳-۱۶
۲۷	روابط و قابلیت ردیابی موارد کاربرد	۳-۳-۱۷
۲۸	ورودی ها، خروجی ها و اعتبارسنجی	۳-۴
۲۸	امنیت	۳-۵
۲۹	جمع بندی فصل	۳-۶

۳۰	معماری و طراحی	۴
۳۰	نمای کلی	۴-۱
۳۰	نمودار معماری	۴-۲
۳۱	مدل داده	۴-۳
۳۲	مرزهای خارجی	۴-۴
۳۲	جمع بندی فصل	۴-۵

۳۳	پیاده سازی	۵
۳۳	مسیرهای بک اند	۵-۱
۳۳	لایه Repository	۵-۲
۳۴	Serializer و قرارداد پاسخ	۵-۳
۳۴	فرانت اند	۵-۴
۳۴	جریان های اصلی پیاده سازی	۵-۵
۳۵	بخش های فنی	۵-۶
۳۶	نمونه کد	۵-۷
۳۶	جمع بندی فصل	۵-۸

۳۷	آزمون و ارزیابی	۶
----	-----------------	---

۳۷	۶-۱	راهنبرد آزمون
۳۷	۶-۲	نتایج اجرا
۳۸	۶-۳	جدول آزمون ها
۳۹	۶-۴	محدودیت های آزمون
۳۹	۶-۵	جمع بندی فصل

۴۰	۷	نتایج و بحث
۴۰	۷-۱	ویژگی های پیاده سازی شده
۴۰	۷-۲	نمونه خروجی API
۴۱	۷-۳	نقاط قوت
۴۱	۷-۴	چالش ها و بدهی فنی
۴۱	۷-۵	بحث
۴۲	۷-۶	جمع بندی فصل

۴۳	۸	نتیجه گیری و کارهای آینده
۴۳	۸-۱	جمع بندی کار انجام شده
۴۳	۸-۲	دستاوردهای اصلی
۴۴	۸-۳	محدودیت های فعلی
۴۴	۸-۴	کارهای آینده
۴۴	۸-۵	جمع بندی فصل

۴۵	منابع
۴۷	الف راهنمای اجرا و شواهد تکمیلی
۴۷	۱-الف اجرای محلی
۴۷	۲-الف اجرای آزمون بک اند
۴۸	۳-الف مسیرهای مهم

فهرست علائم و نشانه‌ها

نماد	شرح
API	رابط برنامه‌نویسی کاربردی
DRF	Django REST Framework
SPA	برنامه تک‌صفحه‌ای
RTL	راست‌به‌چپ
ORM	نگاشت شیء-رابطه

فهرست جداول

۷	نیازمندی‌های اصلی پروژه	۳-۱
۹	خلاصه موارد کاربرد پروژه	۳-۳
۲۷	ردیابی موارد کاربرد به نیازمندی‌های پروژه	۳-۵
۳۱	گروه‌های داده اصلی	۴-۱
۳۴	جریان‌های اصلی در پیاده‌سازی	۵-۱
۳۵	بخش‌های فنی و نقش آن‌ها در پروژه	۵-۳
۳۸	خلاصه آزمون‌های اجراشده	۶-۱
۴۸	مسیرهای مهم برای بررسی دستی سامانه	الف-۱

فهرست نمودارها، عکس‌ها و نقشه‌ها

۸	نمای نقش‌ها و جریان‌های اصلی پروژه	۳-۱
۱۲	نمودار PlantUML مورد کاربرد UC-01	۳-۲
۱۳	نمودار PlantUML مورد کاربرد UC-02	۳-۳
۱۴	نمودار PlantUML مورد کاربرد UC-03	۳-۴
۱۵	نمودار PlantUML مورد کاربرد UC-04	۳-۵
۱۶	نمودار PlantUML مورد کاربرد UC-05	۳-۶
۱۷	نمودار PlantUML مورد کاربرد UC-06	۳-۷
۱۸	نمودار PlantUML مورد کاربرد UC-07	۳-۸
۱۹	نمودار PlantUML مورد کاربرد UC-08	۳-۹
۲۰	نمودار PlantUML مورد کاربرد UC-09	۳-۱۰
۲۱	نمودار PlantUML مورد کاربرد UC-10	۳-۱۱
۲۲	نمودار PlantUML مورد کاربرد UC-11	۳-۱۲
۲۳	نمودار PlantUML مورد کاربرد UC-12	۳-۱۳
۲۴	نمودار PlantUML مورد کاربرد UC-13	۳-۱۴
۲۵	نمودار PlantUML مورد کاربرد UC-14	۳-۱۵
۲۶	نمودار PlantUML مورد کاربرد UC-15	۳-۱۶
۳۱	معماری سطح بالای پروژه	۴-۱

فصل ۱

مقدمه

در شروع گزارش، بهتر است پروژه را از همان جایی ببینیم که کاربر با آن روبه‌رو می‌شود: یک محیط فارسی برای پیدا کردن، بررسی، انتشار و مصرف API. اگر این تصویر روشن باشد، توضیحات فنی فصل‌های بعد هم معنا پیدا می‌کنند؛ چون هر مسیر، جدول و تصمیم معماری در نهایت باید به یک تجربه قابل فهم برای کاربر یا توسعه‌دهنده برسد.

1-1 زمینه پروژه

IranAPI Hub یک پلتفرم فارسی برای معرفی، جستجو، انتشار و مصرف API است. فضای پروژه به بازارچه‌های API نزدیک است؛ جایی که کاربر قبل از استفاده از یک سرویس، می‌تواند آن را بشناسد، مستنداتش را بخواند، پلن‌ها را مقایسه کند و بعد از ورود، مصرف و دسترسی‌های خود را مدیریت کند. این نگاه با مدل رایج API Hub و بازارچه‌های API هم‌خوان است [۱۱]. از سمت فنی، پروژه روی ترکیب Python، Django REST Framework، React، Vite، MongoDB و Docker Compose شکل گرفته است [۱، ۲، ۳، ۴، ۶، ۹، ۸]. این ترکیب برای سامانه‌ای مناسب است که هم به API ساخت‌یافته نیاز دارد و هم به رابط کاربری سریع، فارسی و قابل توسعه.

2-1 بیان مسئله

کاربر چنین سامانه‌ای معمولاً دنبال یک فهرست ساده از سرویس‌ها نیست. او می‌خواهد سریع بفهمد هر API چه کاری انجام می‌دهد، مستندات هر Endpoint چگونه نوشته شده، چه پلن‌هایی دارد و بعد از ورود، وضعیت مصرف، کلیدها، اشتراک و سازمان خود را کجا می‌تواند ببیند. از طرف دیگر، توسعه‌دهنده هم فقط مصرف‌کننده نیست؛ باید بتواند API تازه منتشر کند، یک درخواست نمونه بفرستد، جریان کاری خود را در Studio تنظیم کند و پروژه فشرده خود را پیش از استقرار تحلیل و در محیط ایزوله اجرا کند.

مسئله اصلی از همین جا شروع می‌شود: تجربه کاربر نباید پخش و تکه‌تکه باشد. وقتی مستندات، پرداخت، احراز هویت، مصرف و ابزار تست هر کدام در فضای جداگانه‌ای قرار بگیرند،

کاربر برای رسیدن به نتیجه چند بار مسیر ذهنی خود را عوض می‌کند. این پروژه همین اجزا را در یک تجربه منسجم جمع می‌کند؛ تجربه‌ای که برای کاربر عمومی قابل فهم باشد و برای توسعه‌دهنده هم کاربرد عملی داشته باشد.

3-1 اهداف

- با این نگاه، هدف‌های اصلی پروژه چنین تعریف شده‌اند:
- ارائه کاتالوگ API با جستجو، دسته‌بندی، امتیاز، پلن و مستندات.
 - فراهم کردن ثبت‌نام، ورود، خروج و نشست کاربر با مدل رایج احراز هویت وب.
 - نمایش Dashboard توسعه‌دهنده شامل پروفایل، API Key، مصرف، سازمان و اشتراک.
 - پشتیبانی از انتشار API، اجرای نمونه درخواست در Caller و ساخت جریان در Studio.
 - تولید اسکلت پروژه، تحلیل ایستای آرشیو و صف‌بندی استقرار محدودشده برای پروژه‌های توسعه‌دهنده.
 - ارائه قرارداد OpenAPI برای خوانایی بهتر مسیرها و امکان اتصال ابزارهای دیگر [۱۰].
- این هدف‌ها فقط به ظاهر سامانه مربوط نیستند. کاتالوگ و مستندات بخش عمومی محصول را می‌سازند؛ ورود، اشتراک و مصرف بخش خصوصی را شکل می‌دهند؛ و OpenAPI کمک می‌کند قرارداد سرویس هم برای انسان خوانا باشد و هم برای ابزارهای بیرونی قابل استفاده بماند.

4-1 دامنه و محدودیت‌ها

دامنه گزارش همان نسخه فعلی پروژه است: وب‌اپلیکیشن فارسی، API بک‌اند، اجرای محلی و جریان‌های اصلی کاربر. اجرای محلی با Docker Compose هم در همین دامنه قرار می‌گیرد، چون برای بالا آوردن چند سرویس کنار هم طراحی شده است [۸].

در کنار قابلیت‌ها، چند محدودیت هم باید صریح گفته شود. تنظیمات تولیدی هنوز کامل نیست، بخشی از رفتارها برای محیط توسعه مناسب‌ترند و پرداخت واقعی و اتصال دارای credential به سرویس‌های بیرونی نیاز به سخت‌سازی دارد. آزمون مرورگر اجرا شده، اما باید در چرخه یکپارچه‌سازی مداوم هم منظم شود. بنابراین گزارش، پروژه را به عنوان یک نمونه کامل و قابل توسعه بررسی می‌کند، نه محصولی که همین حالا برای بهره‌برداری عمومی آماده باشد.

5-1 ذی‌نفعان

ذی‌نفعان اصلی پروژه شامل کاربر مهمان، توسعه‌دهنده احرازشده، مدیر سیستم، ارائه‌دهنده ورود اجتماعی، سرویس مقصد API و بازارهای بیرونی مانند RapidAPI هستند [۱۱]. انتظار این گروه‌ها یکسان نیست: مهمان به شناخت سریع سرویس نیاز دارد، توسعه‌دهنده به ابزار و داده خصوصی، مدیر به کنترل سامانه، و سرویس‌های بیرونی به قراردادی قابل اعتماد.

6-1 سازمان‌دهی گزارش

ادامه گزارش به همین ترتیب پیش می‌رود: ابتدا فناوری‌های استفاده‌شده معرفی می‌شوند، سپس نیازمندی‌ها و معماری بررسی می‌شوند. بعد از آن، پیاده‌سازی، آزمون، نتایج و در پایان مسیر ادامه کار توضیح داده می‌شود.

7-1 جمع‌بندی فصل

در این فصل روشن شد که پروژه فقط یک تمرین بک‌اند یا یک رابط کاربری ساده نیست؛ یک پلتفرم اشتراک‌گذاری و مصرف API است. به همین دلیل، گزارش باید هم تجربه کاربر را جدی بگیرد و هم قرارداد فنی میان فرانت‌اند، بک‌اند و داده را دقیق توضیح دهد.

فصل ۲

فناوری‌ها و پیشینه

انتخاب فناوری در این پروژه فقط بر اساس رایج بودن ابزارها انجام نشده است. هر بخش باید با نقش خود در محصول هماهنگ باشد: بک‌اند باید قرارداد API را روشن نگه دارد، فرانت‌اند باید تجربه فارسی و سریع بسازد، و اجرای محلی باید برای ارائه و آزمون در دسر اضافه ایجاد نکند. بنابراین معرفی فناوری‌ها در ادامه، بیشتر روی نقش آن‌ها در خود محصول تمرکز دارد.

1-2 Django، Python و Django REST Framework

Python زبان اصلی بک‌اند است؛ زبانی خوانا و پرکاربرد که برای توسعه وب و ساخت سرویس‌های داده‌محور انتخاب مناسبی به شمار می‌آید [۱]. روی این زبان، Django چارچوب اصلی وب را فراهم می‌کند و بخش‌هایی مثل تنظیمات، مسیرها، امنیت پایه، مدیریت کاربران، پنل مدیریت و ابزارهای آزمون را در اختیار پروژه می‌گذارد [۲].

Django REST Framework نقش تکمیل‌کننده همین ساختار را دارد. مفاهیمی مثل Serializer، View، اعتبارسنجی ورودی و پاسخ JSON کمک می‌کنند مسیرهای عمومی و خصوصی سامانه با قرارداد منظم‌تری نوشته شوند [۳]. در این پروژه، همین ترکیب پایه ساخت کاتالوگ، حساب کاربری، اشتراک، مصرف و ابزارهای توسعه‌دهنده است.

2-2 MongoDB و مدل داده

داده‌های این پروژه شکل کاملاً یکسانی ندارند. اطلاعات کاربر، نشست، API، پلن، مصرف و جریان Studio هر کدام جزئیات خودشان را دارند. به همین دلیل، مدل سندی برای این فضا طبیعی‌تر است. MongoDB نیز در داکيومنت رسمی خود همین رویکرد سندمحور را برای ذخیره داده‌های انعطاف‌پذیر توضیح می‌دهد [۹]. در گزارش، وقتی از لایه داده صحبت می‌شود، منظور بخشی است که همین داده‌های محصول را برای مسیرهای بک‌اند آماده می‌کند.

این انتخاب برای بازارچه API منطقی است. مستندات یک سرویس، پلن‌های قیمت‌گذاری، رخدادهای مصرف، جریان‌های Studio و تحلیل آرشيو پروژه همیشه مثل رکوردهای ساده و ثابت رفتار نمی‌کنند. مدل سندی باعث می‌شود این تفاوت‌ها با انعطاف بیشتری نگهداری شوند.

3-2 React، TypeScript و Vite

فرانت‌اند با React ساخته شده است؛ کتابخانه‌ای که برای ساخت رابط‌های کاربری مبتنی بر کامپوننت استفاده می‌شود [۴]. برای پروژه‌ای که صفحه‌های متنوعی مثل کاتالوگ، جزئیات سرویس، Dashboard و Studio دارد، این مدل کمک می‌کند اجزای تکرارشونده جدا و قابل نگهداری بمانند. TypeScript هم به کد سمت کاربر نوع‌دهی اضافه می‌کند تا خطاهای ساده زودتر دیده شوند و قرارداد بین بخش‌های مختلف روشن‌تر بماند [۵].

Vite ابزار توسعه و build فرانت‌اند است. داکيومنت رسمی Vite روی سرعت راه‌اندازی و تجربه توسعه سریع تاکید دارد [۶]. این ویژگی در کار روزمره پروژه مهم است، چون تغییر در صفحه‌ها و کامپوننت‌ها باید سریع دیده شود و توسعه رابط کاربری کند نشود.

4-2 Node.js و ابزارهای فرانت‌اند

اکوسیستم فرانت‌اند پروژه روی Node.js و ابزارهای بسته‌بندی جاوااسکریپت می‌چرخد. Node.js اجرای جاوااسکریپت خارج از مرورگر را فراهم می‌کند و پایه بسیاری از ابزارهای توسعه فرانت‌اند است [۷]. کتابخانه‌هایی مثل TanStack Query، Axios و مجموعه‌های کامپوننتی، کار ارتباط با بک‌اند، مدیریت وضعیت درخواست‌ها و ساخت فرم‌ها و جدول‌ها را ساده‌تر می‌کنند.

در گزارش، نام این ابزارها بدون ترجمه آمده است؛ چون برای بسیاری از آن‌ها معادل فارسی دقیق و جاافتاده‌ای وجود ندارد و ترجمه اجباری، متن را کم‌دقت‌تر می‌کند. همین محیط Node.js برای کلاینت خط فرمان نیز به کار رفته است. کلاینت با ذخیره امن توکن، خروجی جدولی یا JSON و گزینه انتظار برای استقرار، قرارداد نسخه‌دار سامانه را برای اسکریپت‌ها قابل استفاده می‌کند.

5-2 Docker Compose و اجرای محلی

Docker Compose برای تعریف و اجرای چند سرویس کنار هم استفاده می‌شود [۸]. در این پروژه، همین ابزار باعث می‌شود بک‌اند و فرانت‌اند کنار هم بالا بیایند و مسیر ارتباطی مشخصی داشته باشند. این روش برای ارائه، آزمون دستی و توسعه محلی مناسب است، هرچند جای پیکربندی تولیدی را نمی‌گیرد.

در مسیر استقرار پروژه، Docker نقش جداگانه‌ای دارد: آرشیو پس از تحلیل ایستا در کارگر صف استخراج می‌شود، تصویر ساخته می‌شود و کانتینر با محدودیت حافظه، پردازنده، فرایند و قابلیت‌ها اجرا می‌گردد.

6-2 OpenAPI و RapidAPI

OpenAPI برای توصیف قرارداد API به کار می‌رود؛ یعنی مسیرها، ورودی‌ها، خروجی‌ها و پاسخ‌ها را به شکلی می‌نویسد که هم انسان و هم ابزار بتوانند آن را بخوانند [۱۰]. RapidAPI نیز نمونه

شناخته‌شده‌ای از فضای بازارچه API است و برای فهم تجربه کاربر در کشف، مقایسه و مصرف API مرجع خوبی به شمار می‌آید [۱۱].

7-2 جمع‌بندی فصل

پشته فنی پروژه با ماهیت آن هماهنگ است: Django REST Framework برای ساخت API، React برای تجربه کاربری، MongoDB برای داده‌های سندی، Docker Compose برای اجرای محلی، و OpenAPI برای خوانایی قرارداد سرویس. نتیجه این ترکیب، گزارشی است که فقط درباره ابزارها حرف نمی‌زند؛ بلکه نشان می‌دهد هر ابزار چه نقشی در شکل‌گیری محصول دارد.

فصل ۳

تحلیل نیازمندی‌ها

نیازمندی‌های این پروژه از دل تجربه کاربر بیرون می‌آیند. کاربر مهمان می‌خواهد سرویس‌ها را راحت پیدا کند؛ توسعه‌دهنده می‌خواهد بعد از ورود، حساب، مصرف و ابزارهای خود را مدیریت کند؛ مدیر هم باید بتواند سامانه را کنترل و نگهداری کند. بنابراین نیازمندی‌ها فقط یک فهرست اداری نیستند، بلکه توضیح می‌دهند هر بخش از سامانه باید چه مشکلی را حل کند.

1-3 نقش‌های کاربری

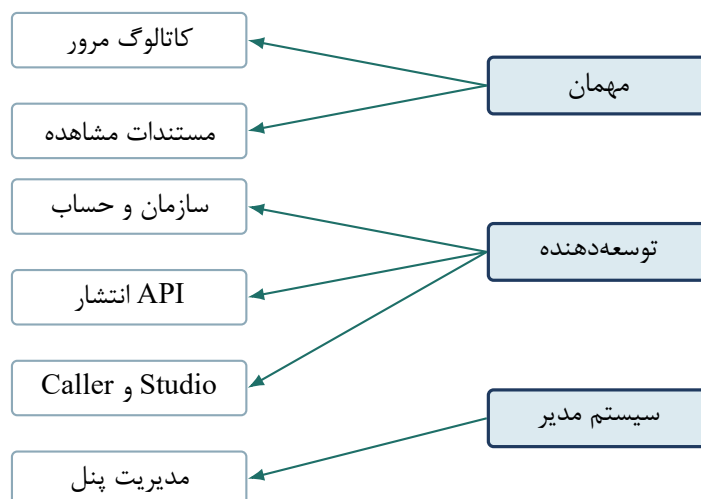
سه نقش اصلی در پروژه دیده می‌شود: مهمان، توسعه‌دهنده احراز شده و مدیر سیستم. مهمان با بخش عمومی کاتالوگ سروکار دارد. توسعه‌دهنده بعد از ورود، وارد فضای کاری خود می‌شود و با Dashboard، انتشار API، Caller و Studio کار می‌کند. مدیر سیستم هم مسئول کنترل کلی و نگهداری داده‌هاست.

مهمان بدون ورود می‌تواند بخش عمومی بازارچه را ببیند؛ این بخش برای معرفی APIها و ایجاد اعتماد اولیه ضروری است. توسعه‌دهنده پس از احراز هویت به امکاناتی می‌رسد که با داده خصوصی او ارتباط دارند، مانند کلید API، سازمان، اشتراک، مصرف، انتشار و ابزارهای آزمایشی. نقش مدیر سیستم بیشتر به نگهداری داده و پشتیبانی از مسیرهای مدیریتی مربوط می‌شود و به همین دلیل از نظر سطح دسترسی حساس‌تر است.

2-3 نیازمندی‌های عملکردی و غیرفعال

جدول ۳-۱ نیازمندی‌ها را در دو گروه اصلی نشان می‌دهد. نیازمندی‌های عملکردی همان کارهایی هستند که کاربر مستقیم با آن‌ها روبه‌رو می‌شود؛ مثل جستجوی API، ورود، انتشار یا مشاهده مصرف. نیازمندی‌های امنیتی و اجرایی کیفیت همین رفتارها را حفظ می‌کنند؛ مثل پنهان کردن کلیدها، پاسخ خطای قابل فهم و اجرای محلی با Docker Compose.

جدول ۳-۱ نیازمندی‌های اصلی پروژه



شکل ۱-۳ نمای نقش‌ها و جریان‌های اصلی پروژه

شناسه	نیازمندی	نوع	اولویت	وضعیت
R1	نمایش فهرست API ها با صفحه‌بندی، جستجو، دسته‌بندی و مرتب‌سازی	عملکردی	زیاد	انجام‌شده
R2	نمایش جزئیات، پلن‌ها، مستندات، endpoint ها و API های مشابه	عملکردی	زیاد	انجام‌شده
R3	ثبت‌نام، ورود، نشست، خروج و کشف ارائه‌دهندگان ورود اجتماعی	عملکردی	زیاد	انجام‌شده
R4	مدیریت کاربر، پروفایل، سازمان، اشتراک، پرداخت دستی و مصرف	عملکردی	زیاد	انجام‌شده
R5	انتشار API توسط کاربر اجراشده	عملکردی	متوسط	انجام‌شده
R6	اجرای درخواست نمونه در Caller و ثبت مصرف	عملکردی	متوسط	انجام‌شده
R7	اعتبارسنجی و ذخیره تعریف جریان Studio همراه ثبت رویداد مصرف، بدون اجرای گره‌ها	عملکردی	متوسط	انجام‌شده
R8	بازگشت خطای یکنواخت برای اعتبارسنجی و دسترسی	غیرفعال	زیاد	انجام‌شده
R9	پنهان‌سازی کلیدها و توکن‌ها در پاسخ‌ها و لاگ‌ها	امنیتی	زیاد	انجام‌شده

شناسه	نیازمندی	نوع	اولویت	وضعیت
R10	اجرای محلی دو سرویس با استقرار Docker Compose	استقرار	متوسط	مستند
R11	تحلیل ایمن آرشیو پروژه فشرده بدون اجرای کد	امنیتی	زیاد	انجام شده
R12	صف بندی و استقرار ایزوله پروژه در محیط محدود شده	استقرار	متوسط	انجام شده
R13	مصرف قرارداد نسخه دار از کلاینت عملکردی خط فرمان	عملکردی	متوسط	انجام شده

اولویت زیاد برای نیازهایی در نظر گرفته شده که نبود آنها هسته محصول را مختل می کند؛ مانند کاتالوگ، احراز هویت، حساب کاربری و امنیت داده حساس. اولویت متوسط برای قابلیت هایی است که ارزش محصول را کامل تر می کنند، اما می توانند در نسخه های بعدی هم رشد کنند؛ مانند Studio، Caller و اجرای محلی.

3-3 موارد کاربرد سامانه

مورد کاربرد، رفتار سامانه را از دید بازیگر و برای رسیدن به یک هدف مشخص توصیف می کند. در این بخش، هر مورد کاربرد علاوه بر نام و بازیگر، با پیش شرط، محرک، جریان اصلی، مسیرهای جایگزین و پس شرط بیان می شود. این قالب کمک می کند مرز میان قابلیت پیاده سازی شده، خطای قابل انتظار و نتیجه نهایی هر تعامل روشن باشد.

جدول ۳-۳ نمای کلی موارد کاربرد را نشان می دهد. مهمان به مسیرهای عمومی دسترسی دارد؛ توسعه دهنده احراز شده علاوه بر آنها می تواند داده و ابزارهای اختصاصی خود را مدیریت کند؛ مدیر سیستم نیز از مسیر مدیریتی جداگانه برای نگهداری داده های سامانه استفاده می کند.

جدول ۳-۳ خلاصه موارد کاربرد پروژه

شناسه	عنوان	بازیگر اصلی	نتیجه مورد انتظار
UC-01	کشف و جستجوی API	مهمان یا توسعه دهنده	یافتن سرویس متناسب با نیاز
UC-02	بررسی جزئیات و مستندات	مهمان یا توسعه دهنده	شناخت قرارداد و شرایط مصرف سرویس
UC-03	ثبت نام، ورود و خروج	مهمان برای ورود؛ کاربر احراز شده برای خروج	ایجاد نشست امن یا پایان دادن به آن
UC-04	مدیریت حساب و سازمان	توسعه دهنده	به روز رسانی هویت، کلید و سازمان ها
UC-05	انتخاب و فعال سازی اشتراک	توسعه دهنده	ثبت یا لغو فرایند خرید و فعال شدن پلن

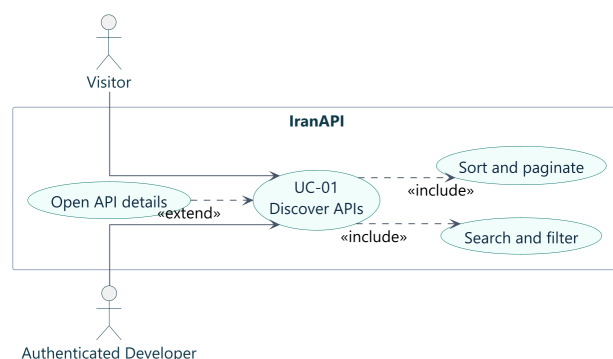
شناسه	عنوان	بازیگر اصلی	نتیجه مورد انتظار
UC-06	انتشار API	توسعه‌دهنده	افزوده‌شدن سرویس به کاتالوگ عمومی
UC-07	ثبت امتیاز برای API	ارائه‌دهنده توسعه‌دهنده مصرف‌کننده	ثبت یا اصلاح نظر عددی کاربر
UC-08	اجرای درخواست در Caller	مهمان یا توسعه‌دهنده	دریافت پاسخ واقعی و، در حالت خصوصی، ثبت مصرف
UC-09	تعریف و ذخیره جریان در Studio	توسعه‌دهنده	ذخیره تعریف جریان و ثبت رویداد مصرف، بدون اجرای گره‌ها
UC-10	ایجاد پروژه آغازین	توسعه‌دهنده	تولید ساختار اولیه پروژه برای مصرف API
UC-11	مشاهده داشبورد و مصرف	توسعه‌دهنده	پایش دسترسی‌ها، اشتراک و سابقه استفاده
UC-12	مدیریت داده‌های زنده سامانه	مدیر سیستم یا توسعه‌دهنده API با دسترسی محدود	مدیریت مجموعه‌های مجاز با کنترل نقش و مالکیت
UC-13	تحلیل آرشیو پروژه	توسعه‌دهنده	تشخیص زبان، چارچوب، مسیرها و آمادگی استقرار
UC-14	استقرار پروژه فشرده	توسعه‌دهنده	ساخت و اجرای ایزوله با نشانی قابل بررسی
UC-15	کار با کاتالوگ و پروژه از خط فرمان	کاربر توسعه‌دهنده فرمان‌های خصوصی	خودکارسازی بررسی و مصرف سرویس

1-3-3 نمودارهای موارد کاربرد

برای جلوگیری از ابهام در نقش‌ها و مرز سامانه، هر مورد کاربرد با یک نمودار مستقل PlantUML در همان صفحه مشخصات نمایش داده شده است. بازیگر اصلی و بازیگران بیرونی خارج از مرز IranAPI قرار دارند و رابطه‌های include و extend فقط رفتارهای واقعاً پیاده‌سازی شده را نشان می‌دهند. فایل‌های منبع نمودارها همراه پروژه نگهداری می‌شوند تا نمودار و پیاده‌سازی همزمان قابل اصلاح باشند.

2-3-3 مورد کاربرد UC-01: کشف و جستجوی API

بازیگر اصلی: مهمان یا توسعه‌دهنده احراز شده.
هدف: یافتن یک API فعال بر اساس مسئله، حوزه کاربرد یا محبوبیت.
پیش شرط: کاتالوگ در دسترس باشد؛ ورود به حساب لازم نیست.
محرک: کاربر صفحه مرور سرویس‌ها را باز می‌کند یا عبارت جستجو و فیلتر را تغییر می‌دهد.



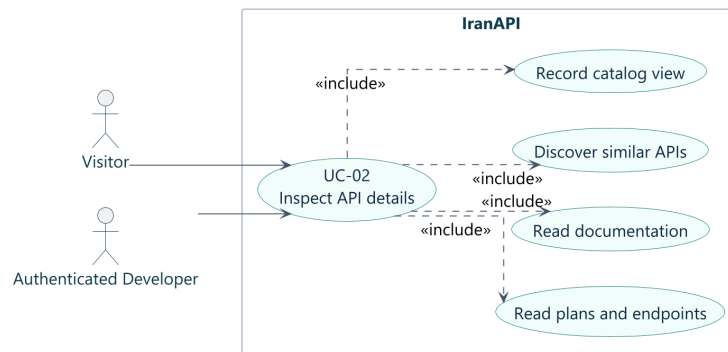
شکل ۳-۲ نمودار PlantUML مورد کاربرد UC-01

جریان اصلی:

۱. سامانه فهرست دسته‌ها و API‌های فعال را از بک‌اند دریافت می‌کند.
 ۲. کاربر می‌تواند عبارت جستجو، دسته، برچسب، وضعیت ویژه یا محبوب و ترتیب نمایش را مشخص کند.
 ۳. بک‌اند شرط‌ها را روی کاتالوگ اعمال و نتیجه را صفحه‌بندی می‌کند.
 ۴. رابط کاربری برای هر نتیجه، اطلاعات خلاصه مانند نام، توضیح کوتاه، دسته، امتیاز و قیمت پایه را نشان می‌دهد.
 ۵. کاربر یکی از نتایج را برای مطالعه بیشتر انتخاب می‌کند.
- مسیرهای جایگزین و خطا:** اگر هیچ سرویس منطبق نباشد، سامانه حالت خالی را نشان می‌دهد و کاربر می‌تواند فیلترها را حذف کند. سرویس غیرفعال در نتیجه عمومی دیده نمی‌شود. درخواست مشاهده سرویس‌های متعلق به کاربر نیز فقط پس از احراز هویت پذیرفته می‌شود.
- پس شرط:** فهرست منطبق با معیارهای کاربر نمایش داده می‌شود؛ تا زمانی که جزئیات یک سرویس باز نشود، داده عملیاتی کاربر تغییر نمی‌کند.

3-3-3 مورد کاربرد UC-02: بررسی جزئیات و مستندات

بازیگر اصلی: مهمان یا توسعه‌دهنده احراز شده.
هدف: ارزیابی فنی و اقتصادی سرویس پیش از استفاده.
پیش شرط: API انتخاب شده فعال و دارای شناسه متنی معتبر باشد.
محرک: کاربر کارت یک سرویس یا پیوند مستقیم صفحه جزئیات را باز می‌کند.



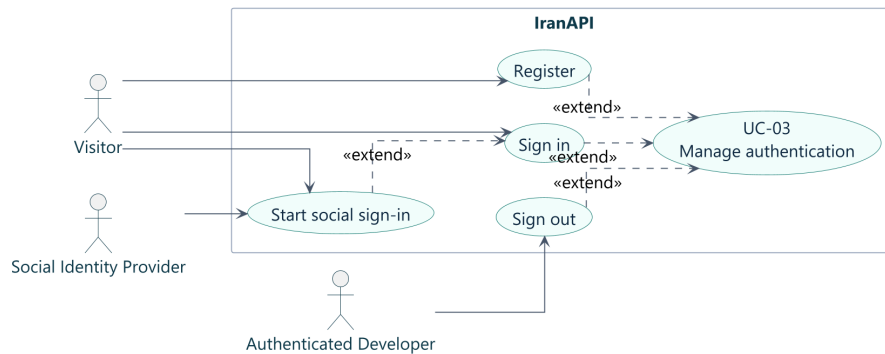
شکل ۳-۳ نمودار PlantUML مورد کاربرد UC-02

جریان اصلی:

۱. سامانه اطلاعات کامل سرویس را با دسته، توضیحات، نشانی پایه و روش احراز هویت دریافت می‌کند.
 ۲. پلن‌های قیمت‌گذاری، مستندات و Endpoint های ثبت شده کنار اطلاعات اصلی نمایش داده می‌شوند.
 ۳. کاربر روش، مسیر، ورودی و توضیح هر Endpoint را بررسی می‌کند.
 ۴. سامانه سرویس‌های مشابه را بر اساس داده کاتالوگ پیشنهاد می‌دهد.
 ۵. شمار مشاهده سرویس برای سنجش میزان توجه کاربران افزایش می‌یابد.
- مسیرهای جایگزین و خطا:** اگر سرویس وجود نداشته یا غیرفعال باشد، پاسخ «یافت نشد» برگردانده می‌شود. نبود پلن، مستندات یا Endpoint مانع نمایش اطلاعات پایه نیست و رابط کاربری برای آن بخش حالت خالی نشان می‌دهد.
- پس شرط:** کاربر اطلاعات لازم برای مقایسه، انتخاب پلن، آزمایش یا کنارگذاشتن سرویس را در اختیار دارد.

4-3-3 مورد کاربرد UC-03: ثبت نام، ورود و خروج

بازیگران اصلی: مهمان برای ثبت نام و ورود؛ توسعه دهنده احراز شده برای خروج.
هدف: ایجاد حساب جدید، بازیابی نشست حساب موجود یا پایان امن نشست.
پیش شرط: برای ثبت نام، نام کاربری و رایانامه باید تکراری نباشند؛ برای ورود، حساب باید از قبل وجود داشته باشد.
محرک: کاربر فرم ثبت نام یا ورود را ارسال می کند، یا فرمان خروج را می زند.



شکل ۳-۴ نمودار PlantUML مورد کاربرد UC-03

جریان اصلی ثبت نام و ورود:

۱. کاربر مشخصات هویتی و گذرواژه را وارد می کند.
۲. سامانه کامل بودن داده، تطابق گذرواژه ها و یکتایی مشخصات را اعتبارسنجی می کند.
۳. در ثبت نام، سند کاربر و پروفایل ساخته می شود؛ در ورود، گذرواژه با داده ذخیره شده تطبیق داده می شود.
۴. بک اند یک نشست پایدار می سازد و شناسه آن را با کوکی HttpOnly به مرورگر می دهد.
۵. اطلاعات عمومی حساب و پروفایل برای راه اندازی فضای خصوصی به رابط کاربری برگردانده می شود.

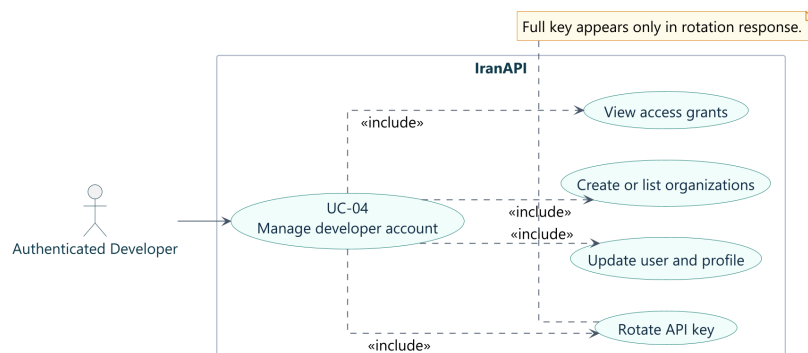
جریان خروج: سامانه نشست سمت سرور را حذف، کوکی مرورگر را منقضی و وضعیت کاربر را مهمان می کند.

مسیرهای جایگزین و خطا: نام کاربری یا رایانامه تکراری، عدم تطابق گذرواژه ها و اطلاعات ورود نادرست با خطای اعتبارسنجی پاسخ داده می شوند. ارائه دهنده ورود اجتماعی فقط در صورت پیکربندی و فعال بودن قابل استفاده است؛ ارائه دهنده ناشناخته یا غیرفعال کاربر را وارد سامانه نمی کند.

پس شرط: در موفقیت، نشست احراز شده فعال است؛ پس از خروج، مسیرهای خصوصی دوباره نیازمند ورود هستند.

5-3-3 مورد کاربرد UC-04: مدیریت حساب، کلید و سازمان

بازیگر اصلی: توسعه‌دهنده احراز شده.
هدف: نگهداری اطلاعات حساب و منابع هویتی مرتبط با فعالیت توسعه‌دهنده.
پیش شرط: نشست یا توکن معتبر وجود داشته باشد.
محرک: کاربر صفحه حساب یا سازمان‌ها را باز می‌کند یا درخواست تغییر می‌فرستد.



شکل ۵-۳ نمودار PlantUML مورد کاربرد UC-04

جریان اصلی:

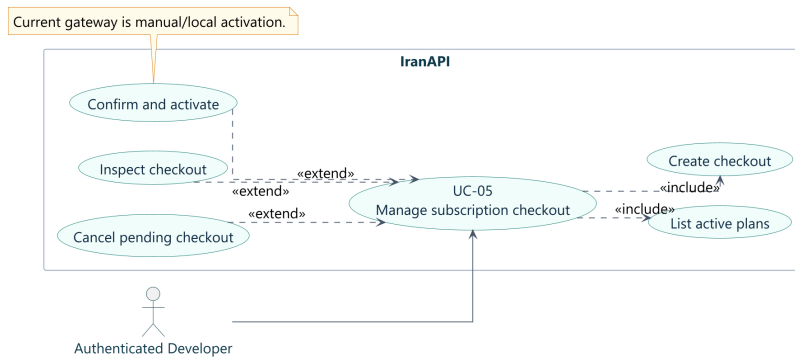
۱. سامانه اطلاعات کاربر و پروفایل را با حذف یا پوشاندن مقادیر حساس نمایش می‌دهد.
۲. کاربر مشخصات مجاز مانند نام، نام خانوادگی یا رایانامه را اصلاح می‌کند.
۳. در صورت نیاز، کاربر کلید دسترسی جدید می‌سازد؛ مقدار کامل کلید فقط در پاسخ همان عملیات نمایش داده می‌شود.
۴. در مشاهده‌های بعدی فقط پیش‌نمایش پوشانده شده کلید برمی‌گردد.
۵. کاربر سازمانی با نام و ناحیه ایجاد می‌کند یا فهرست سازمان‌های متعلق به خود را می‌بیند.
۶. سامانه مجوزهای دسترسی فعال کاربر به API ها و پلن‌های مربوط را نیز قابل مشاهده می‌کند.

مسیرهای جایگزین و خطا: رایانامه تکراری یا داده نامعتبر ذخیره نمی‌شود. درخواست بدون احراز هویت رد می‌شود. روش قدیمی تولید کلید غیرفعال است و چرخش کلید باید از مسیر امن جدید انجام شود.

پس شرط: پروفایل، کلید یا سازمان در مالکیت همان کاربر به‌روزرسانی می‌شود و راز کامل در پاسخ‌های عادی افشا نمی‌شود.

6-3-3 مورد کاربرد UC-05: انتخاب و فعال سازی اشتراک

بازیگر اصلی: توسعه دهنده احراز شده.
هدف: انتخاب پلن اشتراک و مدیریت فرایند پرداخت دستی شبیه سازی شده.
پیش شرط: پلن انتخاب شده فعال باشد و کاربر نشست معتبر داشته باشد.
محرک: کاربر در صفحه قیمت گذاری یک پلن را انتخاب می کند.



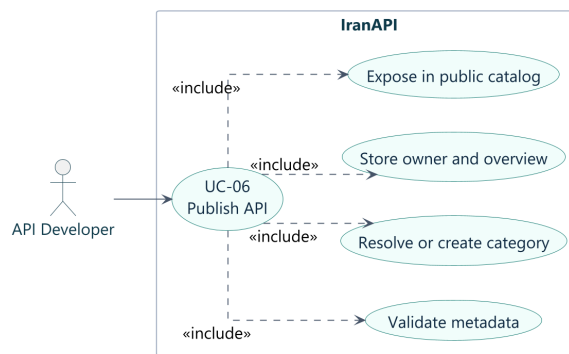
شکل ۳-۶ نمودار PlantUML مورد کاربرد UC-05

جریان اصلی:

۱. سامانه فهرست پلن های قابل خرید را نمایش می دهد.
 ۲. کاربر پلن را انتخاب و یک رکورد Checkout ایجاد می کند.
 ۳. جزئیات مبلغ، وضعیت و زمان اعتبار فرایند خرید نمایش داده می شود.
 ۴. کاربر تأیید را انجام می دهد.
 ۵. سامانه فرایند خرید را تکمیل و اشتراک فعال را برای کاربر ثبت می کند.
 ۶. صفحه حساب، پلن و وضعیت اشتراک جاری را نمایش می دهد.
- مسیرهای جایگزین و خطا:** کاربر می تواند پیش از تکمیل، فرایند خرید را لغو کند. پلن یا فرایند خرید ناموجود، منقضی یا متعلق به کاربر دیگر پذیرفته نمی شود. تأیید دوباره عملیات تکمیل شده، اشتراک تکراری ایجاد نمی کند.
- پس شرط:** فرایند خرید لغوشده باقی می ماند یا اشتراک انتخابی به شکل فعال در حساب ثبت می شود.

7-3-3 مورد کاربرد UC-06: انتشار API

بازیگر اصلی: توسعه‌دهنده ارائه‌دهنده سرویس.
هدف: معرفی یک سرویس تازه و قابل کشف کردن آن در کاتالوگ.
پیش شرط: کاربر وارد حساب شده و اطلاعات فنی سرویس را در اختیار داشته باشد.
محرک: کاربر فرم انتشار را تکمیل و ارسال می‌کند.



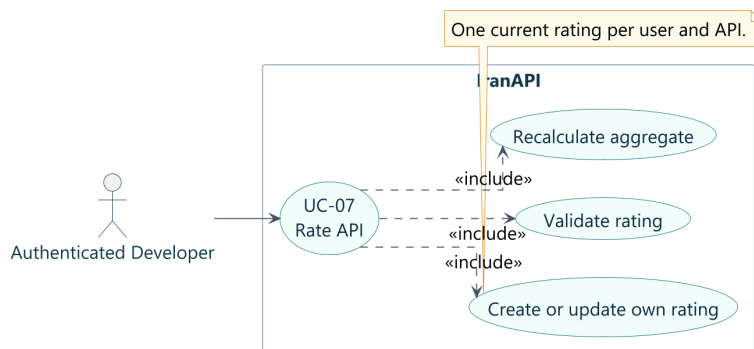
شکل ۷-۳ نمودار PlantUML مورد کاربرد UC-06

جریان اصلی:

۱. کاربر نام، توضیح، نشانی پایه، نشانی مستندات، دسته، روش احراز هویت و برچسبها را وارد می‌کند.
 ۲. سامانه قالب داده و فیلدهای الزامی را اعتبارسنجی و برچسبها و روش احراز هویت را یکدست می‌کند.
 ۳. اگر دسته وارد شده وجود نداشته باشد، دسته‌ای با شناسه متنی مناسب ساخته می‌شود.
 ۴. سند API با شناسه یکتا، مالک فعلی و وضعیت منتشر شده ایجاد می‌شود.
 ۵. توضیح یا پیوند مستندات به عنوان مدرک آغازین سرویس ثبت می‌شود.
 ۶. سرویس تازه در کاتالوگ عمومی و فهرست سرویس‌های متعلق به کاربر قرار می‌گیرد.
- مسیرهای جایگزین و خطا:** درخواست مهمان رد می‌شود. نبود فیلد الزامی، نشانی نامعتبر یا روش احراز هویت غیرمجاز باعث خطای اعتبارسنجی می‌شود و هیچ سرویس ناقصی منتشر نمی‌گردد.
- پس شرط:** یک API فعال و قابل جستجو با رابطه مالکیت مشخص در کاتالوگ ذخیره شده است.

8-3-3 مورد کاربرد UC-07: ثبت یا اصلاح امتیاز

بازیگر اصلی: توسعه‌دهنده احراز شده.
هدف: ثبت ارزیابی عددی از کیفیت یک سرویس و کمک به تصمیم کاربران دیگر.
پیش شرط: کاربر وارد شده و API مورد نظر فعال باشد.
محرک: کاربر در صفحه جزئیات، مقدار امتیاز را ارسال می‌کند.



شکل ۸-۳ نمودار PlantUML مورد کاربرد UC-07

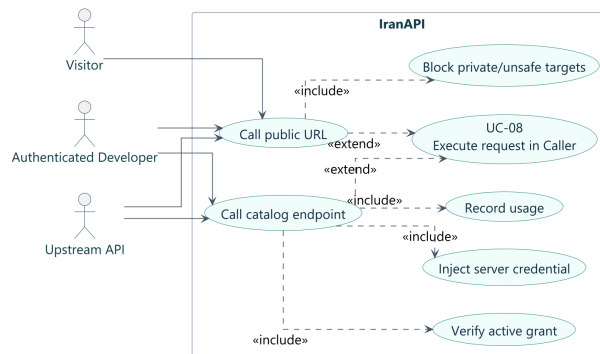
جریان اصلی:

۱. سامانه معتبر بودن دامنه امتیاز را بررسی می‌کند.
 ۲. اگر کاربر قبلاً به این سرویس امتیاز نداده باشد، رکورد تازه ایجاد می‌شود.
 ۳. اگر امتیاز قبلی وجود داشته باشد، همان رکورد به‌روزرسانی می‌شود.
 ۴. میانگین و تعداد امتیازها دوباره محاسبه و همراه امتیاز خود کاربر برگردانده می‌شود.
- مسیرهای جایگزین و خطا:** امتیاز خارج از دامنه، سرویس ناموجود یا درخواست مهمان پذیرفته نمی‌شود. هر کاربر برای هر سرویس فقط یک امتیاز جاری دارد.
پس شرط: امتیاز کاربر و شاخص جمعی سرویس به‌روز شده‌اند.

9-3-3 مورد کاربرد UC-08: اجرای درخواست در Caller

بازیگر اصلی: مهمان برای درخواست مستقیم عمومی؛ توسعه‌دهنده دارای مجوز برای Endpoint کاتالوگ.

هدف: آزمایش یک درخواست واقعی و مشاهده کد وضعیت، زمان پاسخ، نوع محتوا و بدنه پاسخ.
پیش شرط: در حالت کاتالوگ، کاربر باید نشست معتبر و مجوز دسترسی فعال به سرویس داشته باشد؛ تنظیمات ارائه‌دهنده و Endpoint نیز باید در سرور ثبت شده باشند.
محرک: کاربر روش، مقصد، پارامترها و در صورت نیاز بدنه را در Caller وارد و درخواست را اجرا می‌کند.



شکل ۹-۳ نمودار PlantUML مورد کاربرد UC-08

جریان اصلی حالت کاتالوگ:

۱. سامانه داده درخواست را اعتبارسنجی و API و Endpoint ثبت شده را پیدا می‌کند.
۲. روش درخواست باید دقیقاً با روش ثبت شده برای Endpoint برابر باشد.
۳. اعتبار زمانی مجوز دسترسی کاربر بررسی می‌شود.
۴. بک‌اند راز ارائه‌دهنده را در سمت سرور به درخواست اضافه و مقصد را به ارائه‌دهنده مجاز محدود می‌کند.
۵. درخواست با محدودیت زمان، اندازه و تغییر مسیر اجرا می‌شود.
۶. پاسخ پالایش شده به کاربر برمی‌گردد و رکورد مصرف شامل وضعیت، تأخیر و اندازه پاسخ ذخیره می‌شود.

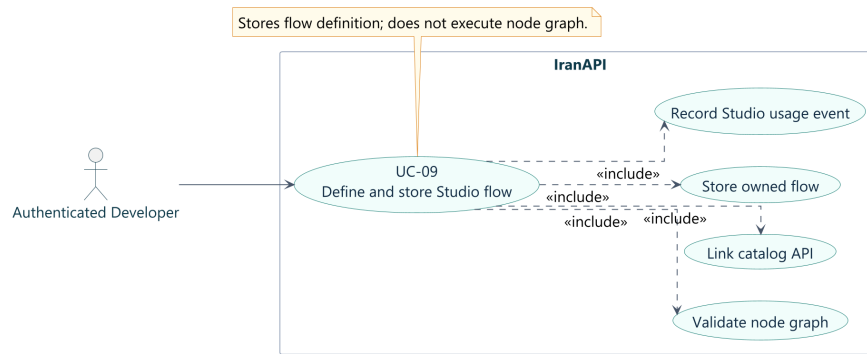
جریان جایگزین عمومی: مهمان می‌تواند یک نشانی عمومی را مستقیم آزمایش کند. در این حالت راز مدیریت شده سرور تزریق نمی‌شود و رویداد مصرف حساب نیز ثبت نمی‌گردد. با این حال، نشانی‌های محلی و خصوصی، سرآیندهای خطرناک و مقصدهای نامعتبر مسدود می‌شوند.

مسیرهای خطا: نبود API یا Endpoint، ناهماهنگی روش، مجوز منقضی، نبود پیکربندی ارائه‌دهنده، پایان زمان انتظار یا خطای سرویس بالادستی با پاسخ کنترل شده گزارش می‌شود؛ درخواست نامعتبر در سابقه مصرف ثبت نمی‌شود.

پس شرط: نتیجه فنی آزمایش نمایش داده می‌شود و فقط اجرای معتبر حالت کاتالوگ در تاریخچه مصرف کاربر قرار می‌گیرد.

10-3-3 مورد کاربرد UC-09: تعریف و ذخیره جریان در Studio

بازیگر اصلی: توسعه‌دهنده احراز شده.
هدف: تعریف و ذخیره یک جریان گره‌ای مرتبط با API.
پیش شرط: کاربر وارد شده و سرویس مرجع در کاتالوگ وجود داشته باشد.
محرک: کاربر نام جریان، سرویس، گره‌ها و ناحیه را برای ذخیره ارسال می‌کند.



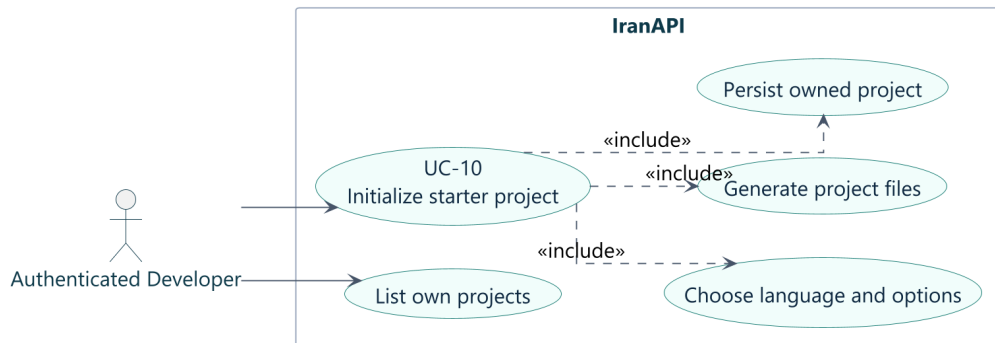
شکل ۱۰-۳ نمودار PlantUML مورد کاربرد UC-09

جریان اصلی:

۱. سامانه ساختار ورودی و گره‌های جریان را اعتبارسنجی می‌کند.
 ۲. سرویس مرجع با شناسه متنی پیدا می‌شود.
 ۳. جریان با مالکیت کاربر، نام، گره‌ها و ناحیه انتخابی ذخیره می‌شود.
 ۴. همزمان یک رویداد مصرف با منبع Studio ثبت می‌گردد.
 ۵. اطلاعات جریان و مصرف به رابط کاربری برمی‌گردد و جریان در فهرست شخصی کاربر دیده می‌شود؛ در این عملیات گره‌ها اجرا نمی‌شوند.
- مسیرهای جایگزین و خطا:** کاربر مهمان، سرویس ناموجود یا ساختار گره نامعتبر باعث رد درخواست می‌شود. فهرست جریان‌ها نیز فقط داده‌های متعلق به کاربر جاری را نمایش می‌دهد.
- پس شرط:** تعریف جریان قابل بازیابی در فضای کار کاربر ذخیره و رویداد مصرف آن ثبت شده است؛ اجرای واقعی گره‌ها جزو این مورد کاربرد نیست.

11-3-3 مورد کاربرد UC-10: ایجاد پروژه آغازین

بازیگر اصلی: توسعه‌دهنده احراز شده.
هدف: ساخت سریع اسکلت یک پروژه مصرف کننده API.
پیش شرط: کاربر نشست معتبر داشته باشد؛ اگر سرویس خاصی انتخاب شده، آن سرویس باید در کاتالوگ موجود باشد.
محرک: کاربر فرمان Init را با نام پروژه و زبان دلخواه ارسال می‌کند.



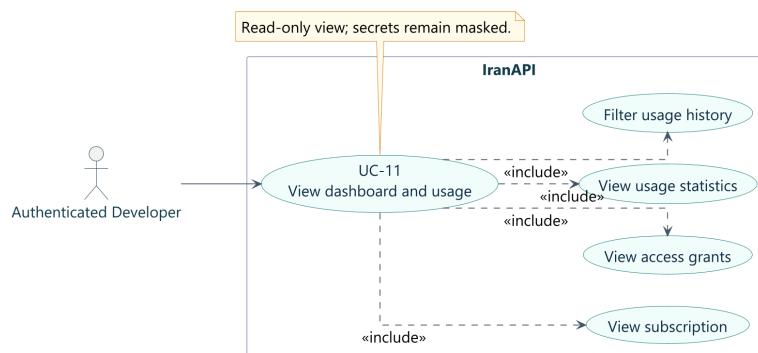
شکل ۱۱-۳ نمودار PlantUML مورد کاربرد UC-10

جریان اصلی:

۱. سامانه زبان‌های آماده و پروژه‌های قبلی کاربر را نمایش می‌دهد.
 ۲. کاربر نام پروژه، زبان، نام بسته، API اختیاری و نیاز به Docker را تعیین می‌کند.
 ۳. بک‌اند ورودی را اعتبارسنجی و قالب متناسب با زبان را انتخاب می‌کند.
 ۴. ساختار فایل‌ها و تنظیمات اولیه تولید و به‌عنوان پروژه متعلق به کاربر ذخیره می‌شود.
 ۵. مشخصات پروژه تولید شده برای نمایش یا دریافت به رابط کاربری برمی‌گردد.
- مسیرهای جایگزین و خطا:** برای زبان خارج از فهرست آماده، قالب عمومی ساخته می‌شود. نام یا داده نامعتبر، سرویس انتخابی ناموجود و درخواست مهمان باعث توقف عملیات می‌شوند.
- پس شرط:** پروژه آغازین قابل بازیابی با تنظیمات انتخابی در حساب کاربر ثبت شده است.

12-3-3 مورد کاربرد UC-11: مشاهده داشبورد، دسترسی و مصرف

بازیگر اصلی: توسعه‌دهنده احراز شده.
هدف: داشتن نمای یکپارچه از حساب، اشتراک، مجوزهای API و الگوی استفاده.
پیش شرط: نشست کاربر معتبر باشد.
محرک: کاربر Dashboard یا صفحه سابقه مصرف را باز می‌کند.



شکل ۱۲-۳ نمودار PlantUML مورد کاربرد UC-11

جریان اصلی:

۱. رابط کاربری اطلاعات حساب، پروفایل، اشتراک جاری و مجوزهای دسترسی را دریافت می‌کند.
 ۲. سامانه شاخص‌های تجمیعی مصرف را برای نمایش در کارت‌های آماری محاسبه می‌کند.
 ۳. سابقه درخواست‌ها با اطلاعات سرویس، منبع رویداد، وضعیت و زمان نمایش داده می‌شود.
 ۴. کاربر می‌تواند سابقه را بر اساس سرویس، منبع یا عبارت جستجو محدود و میان صفحات حرکت کند.
 ۵. اطلاعات حساس مانند کلید کامل در هیچ‌یک از پاسخ‌های داشبورد نمایش داده نمی‌شود.
- مسیرهای جایگزین و خطا:** برای حساب بدون مصرف یا اشتراک، حالت خالی و مقدارهای پایه نمایش داده می‌شود. نشست منقضی کاربر را از دریافت داده خصوصی بازمی‌دارد و نیاز به ورود دوباره دارد.
- پس شرط:** کاربر وضعیت جاری و سابقه فعالیت خود را می‌بیند؛ مشاهده داشبورد به‌تنهایی داده عملیاتی را تغییر نمی‌دهد.

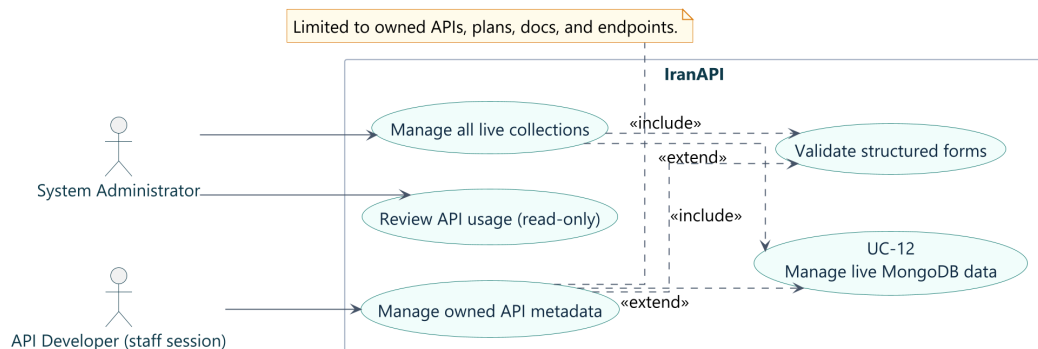
13-3-3 مورد کاربرد UC-12: مدیریت داده‌های زنده سامانه

بازیگران اصلی: مدیر سیستم دارای دسترسی Superuser؛ توسعه‌دهنده API دارای نشست کارمندی با دامنه محدود.

هدف: نگهداری مجموعه‌های زنده MongoDB از مسیر مدیریتی با کنترل نقش، مالکیت و مجموعه.

پیش‌شرط: نشست معتبر پنل مدیریت وجود داشته باشد؛ مدیر سیستم دسترسی کامل دارد و توسعه‌دهنده API فقط به داده متعلق به خود دسترسی می‌گیرد.

محرک: بازیگر یکی از بخش‌های کنسول داده زنده را باز می‌کند.



شکل ۱۳-۳ نمودار PlantUML مورد کاربرد UC-12

جریان اصلی:

۱. پنل بر اساس نقش، فهرست مجموعه‌های مجاز را از داده زنده MongoDB نمایش می‌دهد.
۲. مدیر سیستم می‌تواند همه مجموعه‌های تعریف‌شده را مدیریت کند؛ توسعه‌دهنده API فقط API‌های متعلق به خود و پلن، مستندات و Endpoint‌های وابسته را می‌بیند.
۳. بازیگر یک رکورد مجاز را ایجاد، مشاهده، ویرایش یا حذف می‌کند.
۴. فرم مدیریتی، فیلدهای ساخت‌یافته و داده‌های تو در تو را پیش از ذخیره اعتبارسنجی می‌کند.
۵. تغییر مستقیماً در مجموعه عملیاتی مربوط ذخیره می‌شود تا نتیجه در کاتالوگ یا بخش حساب قابل مشاهده باشد.
۶. مجموعه مصرف API برای مدیر سیستم فقط‌خواندنی است و امکان افزودن، ویرایش یا حذف ندارد.

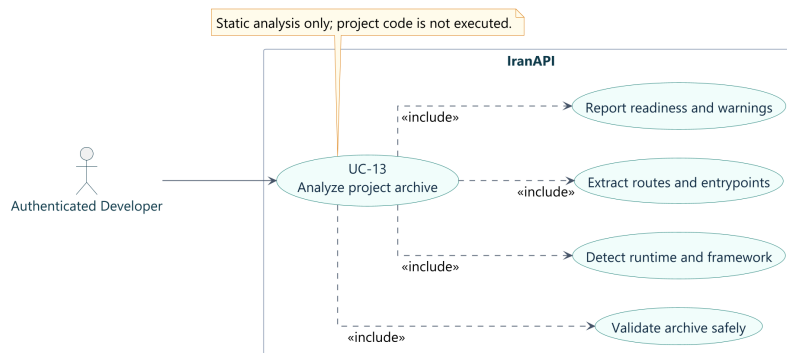
مسیرهای جایگزین و خطا: کاربر عادی و کارمند بدون نقش توسعه‌دهنده API نمی‌توانند کنسول زنده را باز کنند. دسترسی توسعه‌دهنده به سند متعلق به کاربر دیگر رد می‌شود. داده نامعتبر ذخیره نمی‌شود و عملیات تغییر روی مجموعه فقط‌خواندنی پذیرفته نمی‌گردد.

پس‌شرط: داده معتبر در مجموعه زنده ذخیره شده و دامنه مالکیت و سطح دسترسی حفظ شده است.

14-3-3 مورد کاربرد UC-13: تحلیل آرشیو پروژه

بازیگر اصلی: توسعه‌دهنده احراز شده.

هدف: شناخت ساختار یک پروژه فشرده پیش از اجرای آن و دریافت برآورد آمادگی استقرار.
پیش شرط: نشست معتبر وجود داشته باشد و فایل ZIP، TAR، TAR.GZ یا TGZ حداکثر ۲۵ مگابایت باشد.
محرك: کاربر آرشیو پروژه را در ابزار تحلیل بارگذاری می‌کند.



شکل ۱۴-۳ نمودار PlantUML مورد کاربرد UC-13

جریان اصلی:

۱. سامانه نوع آرشیو، تعداد فایل و اندازه باز شده را کنترل می‌کند و مسیرهای ناامن، پیوندها و فایل‌های بیش از حد بزرگ را رد می‌کند.
۲. محتوای متنی بدون اجرای کد بررسی می‌شود تا زبان از میان شش زمان اجرای مدیریت شده تشخیص داده شود.
۳. چارچوب، فایل‌های ورودی، فایل‌های مانیفست، Dockerfile، مسیرهای HTTP و پورت احتمالی استخراج می‌شوند.
۴. هش آرشیو و هشدارهایی مانند نبود نقطه ورود، مسیرهای پویا یا وجود فایل رازدار در نتیجه ثبت می‌شود.
۵. سامانه سازنده پیشنهادی، فرمان شروع، زمان اجرای مقصد و مقدار آمادگی استقرار را برمی‌گرداند.

مسیرهای جایگزین و خطا: آرشیو خالی، خراب، بزرگ‌تر از حد، دارای مسیر پیمایش، پیوند نمادین یا پسوند پشتیبانی نشده با خطای اعتبارسنجی رد می‌شود. زبان ناشناخته یا نبود مسیر HTTP هشدار تولید می‌کند، اما در صورت وجود Dockerfile می‌تواند همچنان قابل استقرار باشد.

پس شرط: نتیجه تحلیل بدون اجرای کد در اختیار کاربر است و فایل خام برای این مسیر ذخیره نمی‌شود.

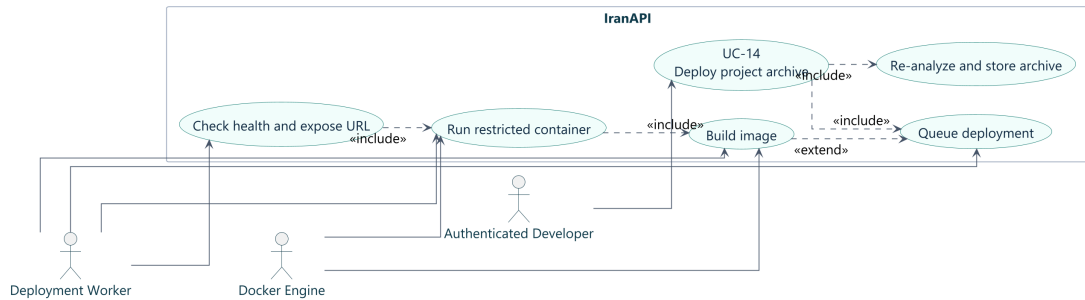
15-3-3 مورد کاربرد UC-14: استقرار پروژه فشرده

بازیگر اصلی: توسعه‌دهنده احراز شده.

هدف: پذیرش، ساخت و اجرای یک پروژه API در محیط جدا شده و قابل پایش.

پیش شرط: آرشیو تحلیل شده باید زمان اجرای پشتیبانی شده، نقطه ورود یا Dockerfile داشته باشد؛ نام پروژه و ناحیه نیز معتبر باشند.

محرک: کاربر آرشیو، نام پروژه و یکی از نواحی مجاز را برای استقرار ارسال می‌کند.



شکل ۱۵-۳ نمودار PlantUML مورد کاربرد UC-14

جریان اصلی:

۱. سامانه آرشیو را دوباره تحلیل و فقط پروژه آماده را می‌پذیرد؛ سپس فایل را در فضای اختصاصی کاربر با اثرانگشت آن ذخیره می‌کند.
 ۲. یک رکورد استقرار با وضعیت queued ساخته می‌شود و کارگر استقرار آن را به وضعیت building می‌برد.
 ۳. کارگر آرشیو را با کنترل مسیر استخراج می‌کند و از Dockerfile موجود یا سازنده زبان محور استفاده می‌کند.
 ۴. تصویر با زمان ساخت محدود ساخته و کانتینر با محدودیت حافظه، پردازنده، فرایند، قابلیت‌های لینوکس و tmpfs اجرا می‌شود.
 ۵. سلامت کانتینر و نگاشت پورت بررسی می‌شود؛ در موفقیت نشانی عمومی، شناسه کانتینر و گزارش ساخت ثبت می‌گردد.
 ۶. در شکست ساخت یا خروج برنامه، وضعیت failed و دلیل خلاصه ذخیره می‌شود و کاربر می‌تواند جزئیات استقرار را ببیند.
- مسیرهای جایگزین و خطا:** نام کوتاه یا طولانی، ناحیه نامعتبر، پروژه آماده نشده، آرشیو خارج از فضای اختصاصی، نبود Docker یا پایان زمان ساخت باعث رد یا شکست کنترل شده می‌شود. استقرار هر کاربر فقط با شناسه همان کاربر قابل مشاهده است.
- پس شرط:** یا استقرار با وضعیت deployed و نشانی قابل بررسی ثبت شده است، یا وضعیت failed همراه دلیل قابل پیگیری باقی می‌ماند.

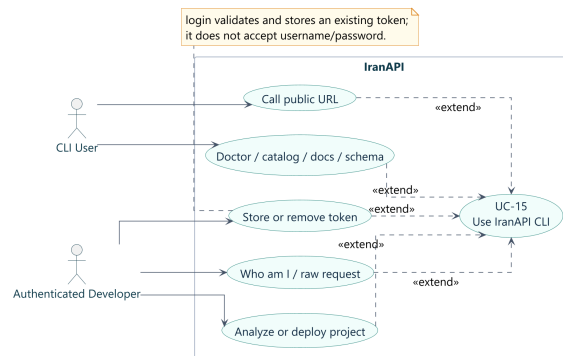
16-3-3 مورد کاربرد UC-15: کار با کاتالوگ و پروژه از خط فرمان

بازیگران اصلی: کاربر CLI برای فرمان‌های عمومی؛ توسعه‌دهنده دارای توکن برای فرمان‌های خصوصی.

هدف: خودکارسازی کشف، مستندسازی، فراخوانی عمومی و چرخه پروژه بدون استفاده مستقیم از مرورگر.

پیش‌شرط: نشانی پایه سامانه تنظیم شده باشد؛ کاتالوگ، مستندات، schema و Caller عمومی به توکن نیاز ندارند، اما فرمان‌های حساب و پروژه به توکن معتبر نیاز دارند.

محرك: کاربر یکی از فرمان‌های login, doctor, apis, docs, call, analyze یا deploy را اجرا می‌کند.



شکل ۱۶-۳ نمودار PlantUML مورد کاربرد UC-15

جریان اصلی:

1. doctor سلامت سامانه، وضعیت پایگاه داده و اعتبار اختیاری توکن را بررسی می‌کند و توکن را پوشانده نمایش می‌دهد.
 2. کاربر با login توکن را پس از اعتبارسنجی در فایل تنظیمات با مجوز محدود ذخیره می‌کند؛ logout آن را حذف می‌کند.
 3. فرمان‌های apis و docs داده صفحه‌بندی‌شده کاتالوگ و مستندات را به شکل جدولی یا JSON برمی‌گردانند.
 4. call درخواست عمومی را از مسیر امن Caller عبور می‌دهد و request برای قرارداد خام سامانه استفاده می‌شود.
 5. analyze نتیجه تحلیل ایستای آرشیو و deploy نتیجه صف یا وضعیت نهایی استقرار را برمی‌گردانند؛ گزینه no-wait برای صف‌بندی بدون انتظار وجود دارد.
- مسیرهای جایگزین و خطا: نبود توکن برای فرمان خصوصی، توکن نامعتبر، نشانی نادرست، پاسخ خطا دار سامانه، فایل بدنه نامعتبر یا پایان زمان انتظار با خطای ساخت‌یافته و کد مناسب گزارش می‌شود. توکن کامل در خروجی چاپ نمی‌شود.
- پس‌شرط: نتیجه عملیات در قالب انسانی یا JSON قابل پردازش در اختیار اسکریپت یا توسعه‌دهنده قرار می‌گیرد.

17-3-3 روابط و قابلیت ردیابی موارد کاربرد

میان موارد کاربرد وابستگی وجود دارد. UC-02 ادامه طبیعی انتخاب سرویس در UC-01 است و UC-07 به صورت اختیاری از صفحه جزئیات انجام می شود. UC-03 پیش نیاز رفتارهای خصوصی مرورگر از UC-04 تا UC-11 و نیز UC-13 و UC-14 است؛ دسترسی مدیر سیستم در UC-12 از ورود پنل مدیریت و شاخه خصوصی UC-15 از توکن انجام می شود. مجوز فعال پیش از اجرای مدیریت شده Caller در UC-08 کنترل می گردد. اجرای معتبر Caller و ذخیره جریان Studio رویدادهایی می سازند که در UC-11 دیده می شوند. UC-13 بخشی از تصمیم گیری UC-14 است و UC-15 مسیرهای عمومی و خصوصی را برای خودکارسازی مصرف می کند.

جدول ۵-۳ ردیابی موارد کاربرد به نیازمندی های پروژه

موارد کاربرد	نیازمندی مرتبط	شاهد رفتاری
UC-01، UC-02، UC-07	UC- R1 و R2	کاتالوگ، جزئیات، مستندات، پیشنهاد مشابه و امتیاز
UC-03	R3، R8 و R9	ثبت نام، نشست HttpOnly، خروج و خطای اعتبارسنجی
UC-04، UC-05، UC-11	UC- R4 و R9	پروفایل، سازمان، اشتراک، دسترسی، مصرف و کلید پوشانده شده
UC-06	R5 و R8	اعتبارسنجی و انتشار سرویس متعلق به کاربر
UC-08	R6، R8 و R9	اجرای محدود شده درخواست، حفاظت راز و ثبت مصرف
UC-09	R7 و R8	اعتبارسنجی گره ها، ذخیره جریان و ثبت مصرف
UC-10	R4	تولید و نگهداری پروژه آغازین در حساب
UC-12	R8 و R9	کنترل نقش مدیر، دامنه مالکیت توسعه دهنده API، فرم معتبر و مجموعه مصرف فقط خواندنی
UC-13	R9 و R11	تحلیل ایستا، محدودیت آرشیو و هشدار فایل های رازدار

موارد کاربرد	نیازمندی مرتبط	شاهد رفتاری
UC-14	R9 و R12	صف ساخت، کانتینر محدودشده و جداسازی پروژه کاربر
UC-15	R9 و R13	فرمان‌های عمومی بدون توکن، فرمان‌های خصوصی با توکن، خروجی جدولی/JSON و پوشاندن راز

4-3 ورودی‌ها، خروجی‌ها و اعتبارسنجی

ورودی‌های اصلی شامل مشخصات حساب، پروفایل، شناسه یا نامک API، فیلترهای جستجو، امتیاز، داده انتشار API، شناسه پلن اشتراک، گره‌های Studio، آرشیو فشرده پروژه، نام پروژه و ناحیه است. در معماری مبتنی بر Django REST Framework، معمولاً Serializer همین داده‌ها را کنترل، پاک‌سازی و به شکل قابل استفاده تبدیل می‌کند [۳].

خروجی‌های اصلی سامانه در قالب JSON بازگردانده می‌شوند و برای مصرف فرانت‌اند طراحی شده‌اند. پاسخ‌های عمومی معمولاً داده کاتالوگ، مستندات، پلن‌ها و وضعیت سلامت را شامل می‌شوند. پاسخ‌های خصوصی، اطلاعاتی مانند پروفایل، سازمان‌ها، کلید پنهان‌شده، اشتراک و تاریخچه مصرف را برمی‌گردانند. جداسازی این دو دسته مهم است، چون داده عمومی برای همه کاربران قابل نمایش است، اما داده خصوصی باید فقط بعد از احراز هویت برگردد.

اعتبارسنجی در این پروژه دو کار اصلی انجام می‌دهد: جلوی ذخیره داده ناقص یا نامعتبر را می‌گیرد و خطا را به پاسخی قابل فهم برای Client تبدیل می‌کند. برای نمونه، هنگام ثبت امتیاز باید مقدار امتیاز معتبر باشد، هنگام انتشار API فیلدهای اصلی باید حاضر باشند و هنگام اجرای Studio ساختار گره‌ها باید قابل پردازش باشد.

5-3 امنیت

احراز هویت باید طوری انجام شود که کاربر بعد از ورود به بخش‌های خصوصی برسد، اما داده حساس او در پاسخ‌ها آشکار نشود. در پروژه‌ای که با API Key، توکن و کوکی کار می‌کند، این موضوع فقط یک جزئیات جانبی نیست؛ بخشی از امنیت اصلی محصول است [۲، ۳].

از دید نیازمندی، امنیت فقط ورود کاربر نیست. کنترل دسترسی، نگهداری نشست، جلوگیری از افشای رازها و محدود کردن خروجی‌های حساس هم بخشی از همین موضوع‌اند. آزمون‌های موجود نشان می‌دهند که پنهان‌سازی داده حساس در خروجی‌ها بررسی شده است. با این حال، برای محیط تولید باید رازهای پیکربندی از فایل تنظیمات خارج شوند و سیاست‌های سخت‌گیرانه‌تری برای کوکی، دامنه مجاز و خطاهای عملیاتی اعمال شود.

6-3 جمع‌بندی فصل

نیازهای پروژه حول کشف عمومی API، فضای کاری توسعه‌دهنده، مدیریت حساب و اشتراک، و امنیت نشست شکل گرفته‌اند. همین نیازها مبنای فصل معماری هستند؛ چون لایه‌های سامانه باید بتوانند نقش‌ها، جریان‌ها و محدودیت‌های امنیتی را پشتیبانی کنند.

فصل ۴

معماری و طراحی

معماری پروژه را می‌توان از مسیر یک درخواست ساده فهمید. کاربر از رابط React شروع می‌کند، درخواست به بک‌اند Django REST Framework می‌رسد، داده اعتبارسنجی می‌شود و سپس لایه داده نتیجه را آماده می‌کند. توضیح معماری در اینجا بر همین مسیر تکیه دارد، بدون این‌که خواننده را درگیر نام‌های داخلی فایل‌ها کند.

1-4 نمای کلی

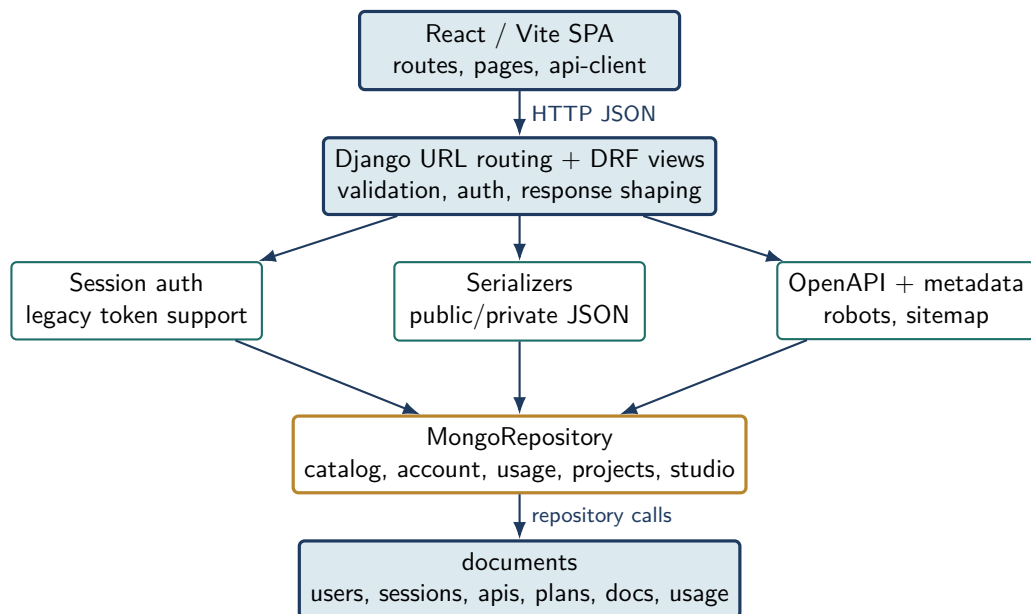
معماری پروژه سه لایه اصلی دارد: رابط کاربری React، بک‌اند Django REST Framework و لایه داده. این مدل با الگوی رایج وب‌اپلیکیشن‌های مدرن هماهنگ است؛ فرانت‌اند چیزی را می‌سازد که کاربر می‌بیند، بک‌اند قرارداد API را مدیریت می‌کند و لایه داده مسئول نگهداری اطلاعات محصول است [۴، ۳، ۹].

در این طراحی، کاربر نهایی بیشتر با یک برنامه تک‌صفحه‌ای روبه‌رو است، اما عملیات داده از مسیر API نسخه‌دار انجام می‌شود. نسخه‌دار بودن مسیرها برای نگهداری آینده اهمیت دارد، چون اجازه می‌دهد قراردادهای جدید اضافه شوند بدون این‌که Client‌های قدیمی ناگهان از کار بیفتند. نمایش پوسته SPA برای مسیرهای ناشناخته فرانت‌اند نیز با مسیریابی سمت Client هماهنگ است.

2-4 نمودار معماری

در شکل ۴-۱، مسیر اصلی ارتباط دیده می‌شود: فرانت‌اند درخواست‌ها را به API بک‌اند می‌فرستد، بک‌اند ورودی را اعتبارسنجی می‌کند، پاسخ را به قالب JSON درمی‌آورد و کار با داده را به لایه Repository می‌سپارد.

در این معماری، هر لایه مسئولیت مشخصی دارد. فرانت‌اند نمایش، فرم‌ها و حرکت کاربر بین صفحه‌ها را مدیریت می‌کند. بک‌اند احراز هویت، اعتبارسنجی و شکل پاسخ را بر عهده دارد. Repository هم جزئیات ذخیره‌سازی و بازیابی داده را پشت یک مرز روشن نگه می‌دارد. این جداسازی کمک می‌کند تغییر در ظاهر کاربر، منطق داده را بی‌دلیل درگیر نکند.



شکل ۴-۱ معماری سطح بالای پروژه

3-4 مدل داده

مدل داده پروژه حول چند گروه اصلی می‌چرخد: هویت کاربر، کاتالوگ API، دسترسی و مصرف، فضای کاری توسعه‌دهنده و اشتراک. این تقسیم‌بندی با مدل سندی MongoDB سازگار است، چون هر گروه می‌تواند ساختار و جزئیات خودش را داشته باشد [۹].

جدول ۴-۱ گروه‌های داده اصلی

گروه	مجموعه‌ها و مسئولیت
هویت	users, sessions و legacy_tokens برای کاربر، نشست و توکن سازگار
کاتالوگ	categories, apis, pricing_plans, documentations و api_endpoints و api_ratings
مصرف	access_grants و api_usage برای مجوز دسترسی و رخدادهای مصرف

گروه	مجموعه‌ها و مسئولیت
فضای کاری	organizations و studio_flows برای امکانات توسعه‌دهنده
پروژه و استقرار	api_projects و project_deployments برای تحلیل، صف ساخت، وضعیت و نشانی اجرای پروژه
اشتراک	subscription_plans، user_subscriptions و subscription_checkouts

گروه هویت پایه دسترسی کاربر را می‌سازد و به نشست‌ها و توکن‌های سازگار قدیمی متصل است. گروه کاتالوگ محتوای عمومی بازارچه را نگه می‌دارد؛ یعنی دسته‌ها، خود API‌ها، پلن قیمت‌گذاری، مستندات، نقاط انتهایی و امتیازها. گروه مصرف و اشتراک داده‌هایی را ثبت می‌کند که برای داشبورد، کنترل دسترسی و گزارش مصرف لازم‌اند. گروه فضای کاری نیز قابلیت‌های مخصوص توسعه‌دهنده مانند سازمان و جریان Studio را پشتیبانی می‌کند. گروه پروژه و استقرار، مشخصات پروژه‌های تولیدشده، اثرانگشت آرشیو، تحلیل ایستا، صف ساخت، گزارش اجرا و نشانی سرویس را نگه می‌دارد؛ محتوای آرشیو در فضای اختصاصی کاربر ذخیره می‌شود.

4-4 مرزهای خارجی

مرزهای خارجی پروژه شامل ارائه‌دهندگان ورود اجتماعی، سرویس مقصد API، موتور Docker و بازارهای بیرونی مانند RapidAPI است [۱۱]. کلاینت خط فرمان نیز مصرف‌کننده بیرونی قرارداد نسخه‌دار به شمار می‌آید. این مرزها باید روشن باشند، چون هر کدام سطحی از ریسک و وابستگی بیرونی وارد سامانه می‌کنند.

شناخت مرز خارجی برای تحلیل ریسک مهم است. ورود اجتماعی به پیکربندی ارائه‌دهنده وابسته است و در صورت نبود auth_url نمی‌تواند کامل اجرا شود. Caller مقصد را با فهرست مجاز و کنترل DNS محدود می‌کند و راز ارائه‌دهنده را در سرور نگه می‌دارد. تحلیل آرشیو بدون اجرای کد انجام می‌شود؛ پس از آن کارگر استقرار با Docker تصویر می‌سازد و کانتینر را با محدودیت منابع و قابلیت‌ها اجرا می‌کند. بنابراین گزارش میان قابلیت پیاده‌سازی‌شده و اتصال تولیدی واقعی تفاوت می‌گذارد.

5-4 جمع‌بندی فصل

معماری پروژه یک SPA متصل به API بک‌اند است. این طراحی برای نسخه فعلی مناسب است و فضای پروژه را به یک پلتفرم واقعی API نزدیک می‌کند. برای استفاده تولیدی، تصمیم نهایی درباره پایگاه داده، مدیریت رازها و اتصال‌های بیرونی باید دقیق‌تر شود.

فصل ۵

پیاده‌سازی

در پیاده‌سازی، مهم‌ترین نکته این است که کاربر از بیرون چه رفتاری می‌بیند و پشت آن رفتار، چه بخش‌هایی با هم کار می‌کنند. به همین دلیل، متن به جای تکیه بر نام فایل‌ها و مسیرهای داخلی، مسیرهای اصلی محصول را توضیح می‌دهد: از کاتالوگ و ورود گرفته تا Dashboard، انتشار API، Studio، Caller و تحلیل و استقرار پروژه.

1-5 مسیرهای بک‌اند

بک‌اند مجموعه‌ای از مسیرهای نسخه‌دار دارد که بخش‌های اصلی محصول را پوشش می‌دهند: سلامت سامانه، قرارداد OpenAPI، ثبت‌نام، ورود، خروج، حساب کاربری، پروفایل، API Key، سازمان‌ها، اشتراک، مصرف، کاتالوگ، مستندات، Endpoint‌ها، Studio، Caller، تولید پروژه، تحلیل آرشیو و صف استقرار. نسخه‌دار بودن مسیرها کمک می‌کند توسعه آینده کنترل‌شده‌تر باشد و Client‌ها با تغییرات ناگهانی روبه‌رو نشوند [۳، ۱۰].

این مسیرها دو سطح تجربه می‌سازند. سطح اول عمومی است و برای مشاهده کاتالوگ، جزئیات API و مستندات استفاده می‌شود. سطح دوم خصوصی است و بعد از ورود کاربر فعال می‌شود؛ مثل Dashboard، مصرف، اشتراک، سازمان و انتشار API. همین جداسازی باعث می‌شود کاربر مهمان با داده خصوصی روبه‌رو نشود و توسعه‌دهنده بعد از ورود، فضای کاری خودش را ببیند.

2-5 لایه Repository

لایه Repository منطق کار با داده را متمرکز می‌کند. ساخت کاربر، نگهداری نشست، چرخش API Key، خواندن کاتالوگ، ثبت مصرف Caller، ذخیره جریان‌های Studio و مدیریت رکوردهای پروژه و استقرار از همین مسیر انجام می‌شود. این شیوه باعث می‌شود View‌ها بیش از حد شلوغ نشوند و جزئیات داده در یک لایه مشخص بماند.

در پروژه‌ای شبیه بازارچه API، این جداسازی اهمیت زیادی دارد. داده‌های حساب، کاتالوگ، مستندات، اشتراک و مصرف شکل یکسانی ندارند. وقتی این بخش‌ها پشت یک Repository قرار می‌گیرند، تغییر در روش ذخیره‌سازی کمتر به بخش‌های دیگر منت می‌کند.

3-5 Serializer و قرارداد پاسخ

Serializer ها مرز امن بین داده داخلی و خروجی قابل نمایش هستند. داده‌ای که داخل سامانه نگهداری می‌شود، همیشه نباید با همان شکل به کاربر برگردد. برای نمونه، مقدار کامل API Key نباید در پاسخ عادی دیده شود و بهتر است فقط نسخه پنهان شده آن نمایش داده شود. این رویکرد با نقش Serializer در Django REST Framework هماهنگ است [۳].

در بخش انتشار API نیز Serializer کمک می‌کند داده ورودی قبل از ثبت بررسی شود: نام، آدرس پایه، روش احراز هویت، دسته و توضیح سرویس باید روشن باشند. نتیجه این کار، پاسخ‌هایی است که برای فرانت‌اند قابل پیش‌بینی‌تر و برای کاربر قابل فهم‌ترند.

4-5 فرانت‌اند

فرانت‌اند همان جایی است که کاربر بیشترین تماس را با محصول دارد: صفحه مرور API ها، جزئیات سرویس، مستندات، قیمت‌گذاری، پرداخت، ورود، ثبت‌نام، Dashboard، سازمان‌ها، انتشار، Caller، Studio، تولید اسکلت پروژه و ابزار تحلیل و استقرار. معماری کامپوننتی React برای چنین تجربه‌ای مناسب است، چون بخش‌های تکرارشونده مثل فرم، کارت، جدول و وضعیت بارگذاری را می‌توان جدا نگه داشت [۴].

ارتباط فرانت‌اند با بک‌اند از طریق Client های مشخص انجام می‌شود. یک گروه درخواست‌ها برای کاتالوگ است، یک گروه برای احراز هویت، و یک گروه برای حساب کاربری و ابزارهای توسعه‌دهنده. این جداسازی باعث می‌شود صفحه‌ها کمتر به جزئیات آدرس‌ها وابسته باشند و نگهداری پروژه ساده‌تر شود.

5-5 جریان‌های اصلی پیاده‌سازی

جدول ۵-۱ نشان می‌دهد هر جریان مهم از نگاه کاربر چه معنایی دارد و در پشت صحنه کدام بخش‌ها را درگیر می‌کند.

جدول ۵-۱ جریان‌های اصلی در پیاده‌سازی

جریان	بخش‌های درگیر	خروجی برای کاربر
مرور کاتالوگ	فرانت‌اند، Client کاتالوگ، بک‌اند و لایه داده	فهرست قابل جستجو و صفحه‌بندی شده API ها
جزئیات API	صفحه جزئیات، مستندات، پلن‌ها و Endpoint ها	شناخت سرویس قبل از مصرف
ورود و نشست	فرم ورود، احراز هویت، کوکی نشست و پاسخ کاربر	دسترسی به بخش خصوصی

جریان	بخش‌های درگیر	خروجی برای کاربر
Dashboard	حساب کاربری، مصرف، اشتراک، سازمان و API Key	مدیریت وضعیت توسعه‌دهنده
انتشار API	فرم انتشار، اعتبارسنجی و ثبت در کاتالوگ	اضافه‌شدن سرویس جدید به محصول
Studio و Caller	ابزار تست، جریان کاری و ثبت مصرف	آزمایش و طراحی جریان‌های مرتبط با API
تحلیل و استقرار پروژه	بارگذاری آرشیو، تحلیل ایستا، صف کارگر و اجرای محدودشده	ارزیابی و انتشار پروژه در محیط
کلاینت خط فرمان	احراز هویت، کاتالوگ، مستندات، فراخوانی عمومی و پروژه	مصرف خودکار قرارداد نسخه‌دار

6-5 بخش‌های فنی

جدول ۳-۵ نقش بخش‌های فنی را با زبان قابل خواندن خلاصه می‌کند، بدون این‌که گزارش به فهرست نام‌های داخلی پروژه تبدیل شود.

جدول ۳-۵ بخش‌های فنی و نقش آن‌ها در پروژه

بخش	نقش در سامانه	اثر در گزارش
مسیریابی بک‌اند	تعریف سطح API و نسخه‌بندی مسیرها	ارتباطی قرارداد توضیح سامانه
Serializer و View	دریافت درخواست، اعتبارسنجی و پاسخ	توضیح رفتار حساب، کاتالوگ و ابزارها
Repository داده	مدیریت داده‌های کاربر، کاتالوگ، مصرف و اشتراک	توضیح مدل عملیاتی داده
رابط کاربری	ساخت صفحه‌ها و جریان حرکت کاربر	توضیح تجربه محصول

بخش	نقش در سامانه	اثر در گزارش
Client فرانت‌اند	ارتباط کنترل‌شده با بک‌اند	توضیح اتصال صفحه‌ها به API
اجرای محلی	بالا آوردن فرانت‌اند و بک‌اند کنار هم	توضیح مسیر ارائه و آزمون دستی
آزمون‌ها	کنترل رفتارهای اصلی و خطاهای مهم	پشتوانه بخش ارزیابی

7-5 نمونه کد

نمونه زیر بخش کوچکی از منطق پنهان کردن داده حساس را نشان می‌دهد.

```

1 SENSITIVE_KEYS = {"password", "api_key", "token", "secret", "
  authorization"}
2
3 def redact_secrets(value):
4     if isinstance(value, dict):
5         return {
6             key: "[REDACTED]" if str(key).lower() in
              SENSITIVE_KEYS else redact_secrets(item)
7         for key, item in value.items()
8     }
9     return value

```

این قطعه کوتاه است، اما برای چنین پروژه‌ای اهمیت زیادی دارد. سامانه با گذرواژه، API Key، توکن و اطلاعات احراز هویت سروکار دارد. اگر این داده‌ها بدون کنترل وارد پاسخ‌ها یا لاگ‌ها شوند، امنیت کاربر آسیب می‌بیند. جایگزین کردن مقدار حساس با [REDACTED] کمک می‌کند خروجی‌ها برای بررسی فنی قابل استفاده باشند، اما رازهای کاربر آشکار نشوند.

8-5 جمع‌بندی فصل

پیاده‌سازی پروژه از چند بخش روشن ساخته شده است: فرانت‌اند برای تجربه کاربر، بک‌اند برای قرارداد API، Serializer برای کنترل ورودی و خروجی، و Repository برای کار با داده. کنار هم قرار گرفتن این بخش‌ها باعث می‌شود پروژه از یک نمونه ساده فراتر برود و به یک پلتفرم واقعی اشتراک‌گذاری API نزدیک شود.

فصل ۶

آزمون و ارزیابی

بعد از پیاده‌سازی، مهم‌ترین پرسش این است که آیا مسیرهای اصلی سامانه واقعاً پایدار کار می‌کنند یا نه. در این بخش، نتیجه آزمون‌های بک‌اند آمده و مشخص شده بخش‌هایی مثل احراز هویت، کاتالوگ، مصرف، امنیت خروجی و قرارداد عمومی API چگونه بررسی شده‌اند.

1-6 راهبرد آزمون

آزمون‌های بک‌اند چند بخش را پوشش می‌دهند: آماده‌بودن برنامه، پنهان‌سازی رازها، اعتبارسنجی داده، احراز هویت، کاتالوگ، مستندات، امتیازدهی، انتشار API، Caller، Studio، سازمان‌ها، اشتراک، پروفایل، تولید پروژه، تحلیل آرشیو، استقرار ایزوله، طرح OpenAPI و مسیرهای متادیتای سایت. چارچوب آزمون در Django برای همین نوع کنترل رفتار سمت سرور استفاده می‌شود [۲]. راهبرد آزمون ترکیبی است. بخشی از آزمون‌ها رفتار زیرساختی را بررسی می‌کنند؛ مانند آماده‌بودن برنامه، شاخص‌ها و ترتیب برنامه‌های نصب‌شده. بخشی دیگر به رفتار محصول مربوط است؛ مانند ثبت‌نام، ورود، مشاهده کاتالوگ، انتشار API و مدیریت اشتراک. گروه سوم آزمون‌ها روی امنیت و قرارداد خروجی تمرکز دارد؛ مانند پنهان‌سازی رازها، محدودبودن مسیرهای خصوصی و معتبر بودن طرح OpenAPI.

2-6 نتایج اجرا

دستور زیر در ریشه پروژه اجرا شد:

```
python manage.py test api
```

نتیجه شمارش آزمون‌های فعلی نشان می‌دهد 99 آزمون بک‌اند تعریف شده است. اجرای کامل آن‌ها با دستور این فصل روی پایگاه ایزوله test_iranapi انجام می‌شود؛ پایگاه روی سرویس واقعی MongoDB به‌صورت خودکار ساخته و بعد از پایان کار حذف می‌شود. این نتیجه نشان می‌دهد بک‌اند در محدوده آزمون‌های موجود پایدار است. البته باید روشن گفت که این موفقیت، همه جنبه‌های محصول را اثبات نمی‌کند. برای ادعای آمادگی تولید، آزمون‌های

بیشتری مانند بار، امنیت، مرورگر و اجرای کامل Docker Compose باید به چرخه ارزیابی اضافه شود.

3-6 جدول آزمون‌ها

جدول ۱-۶ خلاصه آزمون‌های اجراشده

شناسه	مؤلفه	سناریو	انتظار	نتیجه واقعی	وضعیت
T1	امنیت	حذف گذرواژه، کلید و توکن از داده	رازها دیده نشوند	موفق	قبول
T2	مدل‌ها	نامک فارسی یکتا و اعتبار رنگ و امتیاز	خطا برای داده نامعتبر	موفق	قبول
T3	احراز هویت	ثبت نام، ورود، نشست و خروج	نشست ساخته و پاک شود	موفق	قبول
T4	کاتالوگ	فهرست، جزئیات، مستندات و endpointها	داده صفحه‌بندی	موفق	قبول
T5	حساب	پروفایل، کلید API، سازمان و دسترسی	مسیر خصوصی با احراز هویت	موفق	قبول
T6	اشتراک	ساخت، مشاهده، لغو و تأیید پرداخت دستی	وضعیت درست checkout	موفق	قبول
T7	Studio Caller	ثبت مصرف پس از اجرای یا ذخیره جریان Studio	رکورد مصرف ساخته شود	موفق	قبول
T8	طرح و سایت	OpenAPI، robots.txt و sitemap.xml	پاسخ معتبر	موفق	قبول

شناسه	مؤلفه	سناریو	انتظار	نتیجه واقعی	وضعیت
T9	پروژه	تحلیل آرشیو، مسیر امن، صف استقرار و شکست ساخت	رد ورودی ناامن و ثبت وضعیت	موفق	قبول

جدول نشان می‌دهد پوشش آزمون‌ها به یک قابلیت محدود نیست و بیشتر مسیرهای اصلی محصول را لمس می‌کند. وجود آزمون برای مسیرهای OpenAPI و متادیتای سایت نیز اهمیت دارد، زیرا این مسیرها مستقیماً به مصرف‌کنندگان بیرونی و موتورهای جستجو مربوط می‌شوند. آزمون‌های امنیتی هم بررسی می‌کنند داده‌های حساس در پاسخ عادی سامانه آشکار نشوند.

4-6 محدودیت‌های آزمون

آزمون فرانت‌اند با Playwright دوباره اجرا شد. در مجموعه فعلی، 16 آزمون سلامت صفحه و چهار سناریوی رفتاری برای Caller، خزیدن مسیرها و پنل توسعه‌دهنده وجود دارد؛ در مجموع 20 سناریوی انتها به انتها مسیرهای عمومی، ثبت نام، ورود، ابزارهای پروژه، امتیازدهی، خروج و مسیر نامعتبر را پوشش می‌دهند [۱۲]. علاوه بر آن، صفحه‌های اصلی در اندازه دسکتاپ و موبایل به صورت دیداری بررسی شدند.

محدودیت باقی‌مانده این است که آزمون بار، مرورگرهای متعدد و اتصال پولی واقعی به ارائه‌دهندگان انجام نشده است. Caller تا مرز امن credential واقعی بررسی شد: کنترل مجوز، allowlist، حل DNS عمومی و خطای نبود credential تأیید شدند، اما بدون کلیدهای مالک سرویس هیچ پیامک یا تراکنش خارجی ارسال نشد.

5-6 جمع‌بندی فصل

هر 99 آزمون بک‌اند و 20 سناریوی مرورگر در مجموعه فعلی برای اجرا تعریف شده‌اند. ساخت فرانت‌اند، بررسی دیداری، سلامت MongoDB و قراردادهای اصلی نیز تأیید شد؛ آزمون بار و پیکربندی تولیدی همچنان گام بعدی‌اند.

فصل ۷

نتایج و بحث

در این بخش، پروژه از زاویه نتیجه نهایی بررسی می‌شود: چه چیزی آماده شده، کدام بخش‌ها برای ارائه قابل اتکا هستند و چه کارهایی هنوز باید کامل شوند. هدف، ارائه تصویر واقعی‌تری از وضعیت سامانه است، نه فقط آوردن فهرستی از قابلیت‌ها.

1-7 ویژگی‌های پیاده‌سازی شده

ویژگی‌های اصلی پروژه شامل کاتالوگ عمومی، صفحه جزئیات API، مستندات، پلن‌ها، ثبت امتیاز، ثبت‌نام، ورود، نشست، خروج، پروفایل، چرخش API Key، سازمان‌ها، اشتراک، checkout دستی، مصرف، Caller، Studio، تولید پروژه، تحلیل آرشیو، صف‌بندی استقرار ایزوله و انتشار API است. کلاینت خط فرمان نیز همین قرارداد را برای کاتالوگ، مستندات، فراخوانی عمومی و پروژه‌ها مصرف می‌کند. کنار هم قرار گرفتن این بخش‌ها، پروژه را به فضای API Marketplace نزدیک می‌کند [۱۱].

این ویژگی‌ها دو نوع ارزش ایجاد می‌کنند. ارزش نخست برای مصرف‌کننده API است: کشف، مقایسه، مشاهده مستندات و انتخاب پلن. ارزش دوم برای توسعه‌دهنده یا مالک API است: مدیریت حساب، سازمان، کلید، انتشار، مشاهده مصرف و آزمایش جریان‌ها. همین دواگانگی باعث می‌شود پروژه از یک فهرست ساده فراتر برود و به شکل یک بازارچه عملیاتی نزدیک شود.

2-7 نمونه خروجی API

طرح OpenAPI برای معرفی مسیرها و پاسخ‌ها استفاده می‌شود. نمونه زیر پاسخ ساده مسیر سلامت را نشان می‌دهد [۱۰]:

```
1 GET /api/v1/system/health/
2 {
3   "status": "ok",
4   "timestamp": "2026-05-17T00:00:00Z"
5 }
```

این پاسخ کوتاه است، اما برای ارزیابی اهمیت دارد؛ زیرا نشان می‌دهد مسیر نسخه‌دار API فعال است و Client می‌تواند وضعیت سرویس را بدون ورود بررسی کند. وجود زمان در پاسخ نیز برای عیب‌یابی و مانیتورینگ اولیه مفید است.

3-7 نقاط قوت

نقاط قوت پروژه را می‌توان در چند مورد خلاصه کرد: مسیرهای نسخه‌دار، پاسخ خطای یکنواخت، آزمون‌های متعدد، پنهان کردن رازها، پشتیبانی از فارسی، قرارداد OpenAPI و اجرای محلی با Docker Compose.

از نظر نگهداری، مهم‌ترین نقطه قوت پروژه جداسازی لایه‌هاست. مسیرها، Serializerها، لایه داده و Client فرانت‌اند نقش‌های نسبتاً روشن دارند. از نظر تجربه کاربر، پشتیبانی از راست‌به‌چپ و فارسی باعث می‌شود سامانه برای مخاطب ایرانی مناسب‌تر باشد. از نظر توسعه، وجود آزمون‌ها و طرح OpenAPI مسیر ادامه کار و اتصال Clientهای دیگر را ساده‌تر می‌کند.

4-7 چالش‌ها و بدهی فنی

مهم‌ترین چالش، تصمیم‌نهایی درباره مرز داده و آماده‌سازی تولیدی است. تنظیمات توسعه‌ای باید از تنظیمات تولید جدا شود، رازها باید از محیط امن خوانده شوند، ورود اجتماعی به پیکربندی کامل نیاز دارد، و Caller و کارگر استقرار در نسخه تولیدی باید با کنترل خطا، محدودیت زمانی، پایش و ثبت رویداد دقیق به سرویس مقصد و موتور کانتینر متصل شوند.

این چالش‌ها مانع نمایش پروژه نیستند، اما برای بهره‌برداری واقعی باید حل شوند. دواگانگی داده می‌تواند در آینده باعث اختلاف رفتار بین پنل مدیریت و مسیرهای API شود. تنظیمات توسعه‌ای نیز اگر بدون تغییر وارد محیط تولید شوند، خطر امنیتی ایجاد می‌کنند. همچنین پرداخت دستی و اجرای نمونه Caller باید در نسخه تولیدی با سرویس‌های واقعی، کنترل خطا، محدودیت زمانی و ثبت رویداد دقیق جایگزین یا تکمیل شوند.

5-7 بحث

پیاده‌سازی فعلی برای نمایش و آزمون قابلیت‌های اصلی بازارچه API مناسب است. با این حال، برای استفاده تولیدی باید محرمانگی تنظیمات، پایش و پشتیبان‌گیری سرویس MongoDB، اجرای آزمون‌های مرورگر در CI، جداسازی واقعی کارگر استقرار و مسیر پرداخت واقعی دقیق‌تر شود.

سطح بلوغ پروژه را می‌توان «نمونه کامل قابل اجرا برای محیط توسعه» دانست. قابلیت‌های اصلی محصول وجود دارند و آزمون بک‌اند از آن‌ها پشتیبانی می‌کند، اما همه نیازهای یک سامانه تولیدی هنوز کامل نشده‌اند؛ از جمله مدیریت رازها، اتصال‌های بیرونی واقعی، مشاهده‌پذیری، پایداری زیر بار و چرخه انتشار رسمی.

6-7 جمع‌بندی فصل

نتیجه پروژه یک پورتال نسبتاً کامل برای کشف، انتشار و مدیریت API است. بک‌اند در محدوده آزمون‌های فعلی وضعیت خوبی دارد، اما آماده‌سازی تولیدی هنوز کار می‌خواهد. همین موضوع مبنای پیشنهادهای فصل پایانی است.

فصل ۸

نتیجه‌گیری و کارهای آینده

در پایان گزارش، باید روشن باشد که پروژه چه ارزشی ایجاد کرده و چه فاصله‌ای تا یک محصول آماده بهره‌برداری دارد. این بخش فصل‌های قبل را تکرار نمی‌کند؛ بلکه جمع‌بندی می‌کند سامانه امروز در چه وضعیتی قرار دارد و ادامه کار بهتر است از کجا شروع شود.

1-8 جمع‌بندی کار انجام‌شده

IranAPI Hub یک پلتفرم اشتراک‌گذاری API با بک‌اند Django REST Framework و فرانت‌اند React است. سامانه کاتالوگ، مستندات، احراز هویت، Dashboard، مصرف، اشتراک، سازمان، انتشار، Caller، Studio، چرخه تحلیل و استقرار پروژه و کلاینت خط فرمان را پوشش می‌دهد.

در طول گزارش، پروژه از دو زاویه بررسی شد: تجربه کاربر و ساختار فنی. فصل نیازمندی‌ها نقش‌ها و جریان‌ها را مشخص کرد. فصل معماری نشان داد این جریان‌ها چگونه بین فرانت‌اند، بک‌اند، لایه داده و کارگر استقرار تقسیم می‌شوند. فصل پیاده‌سازی رفتار بخش‌های اصلی را توضیح داد و فصل آزمون مجموعه فعلی شامل 99 آزمون بک‌اند و 20 سناریوی مرورگر را ثبت کرد.

2-8 دستاوردهای اصلی

دستاوردهای اصلی پروژه شامل پشتیبانی از زبان فارسی، مسیرهای عمومی و خصوصی منظم، خروجی OpenAPI، تست‌های بک‌اند، ساختار Docker Compose و تفکیک نسبی بین لایه نمای API، سریال‌سازی، داده و رابط کاربری است.

دستآورد مهم دیگر، هماهنگی نسبی بین تجربه کاربر و معماری فنی است. کاربر کاتالوگ، مستندات، حساب و ابزارهای توسعه‌دهنده را می‌بیند و پشت آن، API، Serializer، لایه داده و آزمون قرار گرفته‌اند. همین رابطه باعث می‌شود پروژه فقط نمایشی نباشد و از نظر فنی هم قابل ادامه‌دادن باشد.

3-8 محدودیت‌های فعلی

محدودیت‌های فعلی شامل نیاز به سخت‌سازی تنظیمات تولید، نبود درگاه پرداخت خودکار، نبود کلیدهای واقعی ارائه‌دهندگان برای ارسال درخواست خارجی، نیاز ورود اجتماعی به پیکربندی ارائه‌دهنده و وابستگی استقرار به موتور Docker است. اجرای امن Caller تا کنترل مجوز و credential واقعی آزمایش شده و تحلیل آرشو نیز پیش از اجرای کد انجام می‌شود. این محدودیت‌ها طبیعی‌اند، اما باید در برنامه ادامه کار جدی گرفته شوند. تا زمانی که تنظیمات تولید، مدیریت رازها، پشتیبان‌گیری، پرداخت و اتصال بیرونی دارای credential کامل نشده باشند، بهتر است پروژه به عنوان نسخه آموزشی یا نمونه قابل توسعه معرفی شود، نه محصول تولیدی آماده بهره‌برداری عمومی.

4-8 کارهای آینده

کارهای آینده پیشنهادی:

- انتقال رازها و تنظیمات حساس به متغیرهای محیطی و غیرفعال کردن DEBUG در تولید.
 - پایش و پشتیبان‌گیری عملیاتی MongoDB و آزمون بازیابی داده.
 - اجرای منظم آزمون‌های Playwright و افزودن آن‌ها به فرایند CI.
 - اتصال Caller به سرویس‌های مقصد با کنترل امنیت، timeout و ثبت خطای واقعی.
 - تکمیل پرداخت واقعی، webhook و کنترل سهمیه مصرف.
 - افزودن مستندات نصب تولید، نسخه‌گذاری API و مانیتورینگ عملیاتی.
- ترتیب انجام این کارها بهتر است از ریسک‌های بنیادی شروع شود. ابتدا باید امنیت پیکربندی و رازها سخت‌سازی شود، سپس پشتیبان‌گیری MongoDB و مشاهده‌پذیری عملیاتی اضافه شوند. پس از آن، پرداخت واقعی و اجرای کنترل‌شده Caller با credentialهای مالک سرویس می‌توانند کامل شوند.

5-8 جمع‌بندی فصل

پروژه پایه فنی مناسبی برای یک بازارچه API دارد و آزمون‌های بک‌اند موجود آن با موفقیت اجرا شده‌اند. مسیر توسعه بعدی باید روی سخت‌سازی تولیدی، یکپارچه‌سازی داده و آزمون‌های انتهابه‌انتهای متمرکز شود. با انجام این گام‌ها، سامانه می‌تواند از نمونه آموزشی کامل به محصولی پایدارتر و آماده‌تر برای استفاده واقعی تبدیل شود.

منابع

- [https:// .Python 3 Documentation .Python Software Foundation](https://docs.python.org/3/) [١]
[.docs.python.org/3/](https://docs.python.org/3/)
- [https://docs . Django Documentation .Django Software Foundation](https://docs.djangoproject.com/) [٢]
[.djangoproject.com/](https://docs.djangoproject.com/)
- [https://www . Official Documentation .Django REST Framework](https://www.djangoproject.com/rest-framework/) [٣]
[.django-rest-framework.org/](https://www.djangoproject.com/rest-framework/)
- [.https://react.dev/ .React Documentation .Meta Open Source](https://react.dev/) [٤]
- [https://www.typescriptlang . TypeScript Handbook .Microsoft](https://www.typescriptlang.org/docs/) [٥]
[.org/docs/](https://www.typescriptlang.org/docs/)
- [.https://vite.dev/guide/ .Vite Guide .Vite](https://vite.dev/guide/) [٦]
- [https://nodejs.org/ .Node.js Documentation .OpenJS Foundation](https://nodejs.org/docs/latest/api/) [٧]
[.docs/latest/api/](https://nodejs.org/docs/latest/api/)
- [https://docs.docker . Docker Compose Documentation .Docker](https://docs.docker.com/compose/) [٨]
[.com/compose/](https://docs.docker.com/compose/)
- [https://www.mongodb.com/ .MongoDB Manual .MongoDB](https://www.mongodb.com/docs/manual/) [٩]
[.docs/manual/](https://www.mongodb.com/docs/manual/)
- [https://spec . OpenAPI Specification .OpenAPI Initiative](https://spec.openapis.org/oas/latest.html) [١٠]
[.openapis.org/oas/latest.html](https://spec.openapis.org/oas/latest.html)
- [.https://docs.rapidapi.com/ .RapidAPI Docs .RapidAPI](https://docs.rapidapi.com/) [١١]
- [https://playwright.dev/ .Playwright Documentation .Microsoft](https://playwright.dev/docs/intro) [١٢]
[.docs/intro](https://playwright.dev/docs/intro)

واژه‌نامه

واژه فارسی	برابر انگلیسی
پلتفرم اشتراک‌گذاری API	API Marketplace
احراز هویت نشست	Session Authentication
Legacy Token	Legacy Token
Studio Flow	Studio Flow
مجوز دسترسی	Access Grant

پیوست الف

راهنمای اجرا و شواهد تکمیلی

1-الف اجرای محلی

اجرای محلی پروژه با Docker Compose انجام می‌شود:

```
1 | docker compose up --build
```

در این پیکربندی، فرانت‌اند روی `http://localhost:5173` و بک‌اند روی `http://localhost:8000/api` منتشر می‌شود.

این روش برای بررسی سریع کل سامانه مناسب است، زیرا هر دو بخش اصلی را هم‌زمان بالا می‌آورد. پس از اجرا، کاربر می‌تواند ابتدا مسیر سلامت بک‌اند را بررسی کند و سپس از طریق فرانت‌اند وارد جریان مرور کاتالوگ، ورود، داشبورد و ابزارهای توسعه‌دهنده شود. اگر یکی از سرویس‌ها اجرا نشود، باید لاگ همان سرویس در Docker Compose بررسی شود.

2-الف اجرای آزمون بک‌اند

دستور اجراشده در این گزارش:

```
1 | python manage.py test api
```

در نسخه فعلی، مجموعه آزمون شامل 99 آزمون بک‌اند است. اجرای آن روی پایگاه ایزوله `test_iranapi` انجام می‌شود؛ پایگاه روی MongoDB ساخته و پس از آزمون حذف می‌شود. در صورت موفقیت محیط، خروجی خلاصه باید OK باشد.

این دستور فقط آزمون‌های برنامه `api` را اجرا می‌کند و برای تکرار نتیجه فصل آزمون کافی است. در صورت تغییر کد بک‌اند، اجرای دوباره این دستور باید نخستین کنترل کیفی باشد. برای تغییرات فرانت‌اند، اجرای آزمون‌های مرورگر و ساخت نسخه فرانت‌اند نیز باید به این کنترل اضافه شود.

3-الف مسیرهای مهم

جدول الف-۱ مسیرهای مهم برای بررسی دستی سامانه

مسیر	کاربرد
/api/v1/system/health/	سلامت سامانه
/api/v1/schema/openapi.json	طرح OpenAPI
/api/v1/auth/login/	ورود نشست‌محور
/api/v1/catalog/apis/	فهرست و انتشار API
/api/v1/account/usage/	تاریخچه مصرف
/api/v1/account/studio/flows/	جریان‌های Studio
/api/v1/account/projects/analyze/	تحلیل ایستای آرشیو پروژه
/api/v1/account/projects/deployments/	صف‌بندی و فهرست استقرار پروژه
/api/v1/account/projects/deployments/<slug>/	جزئیات وضعیت، نشانی و گزارش استقرار